AI Exam — Self-Test

70 exam-level applied multiple-choice questions · 10 lectures (L02–L12) · with diagrams

How to use this test

- Work through all 70 questions and record your letter answers on the sheet below (or on paper).
- Each question has exactly **one** correct option. These are **applied** questions — expect to trace algorithms, compute values, and classify scenarios, not just recall definitions.
- When done, check the **Answer Key** at the end (it starts on its own page so you won't see answers while solving). Each answer includes a short explanation.
- Tally your score *per lecture* to see exactly which topics to review.

Answer sheet

Q1 (L02): ____	**Q2** (L02): ____	**Q3** (L02): ____	**Q4** (L02): ____	**Q5** (L02): ____
Q6 (L02): ____	**Q7** (L02): ____	**Q8** (L03): ____	**Q9** (L03): ____	**Q10** (L03): ____
Q11 (L03): ____	**Q12** (L03): ____	**Q13** (L03): ____	**Q14** (L03): ____	**Q15** (L05): ____
Q16 (L05): ____	**Q17** (L05): ____	**Q18** (L05): ____	**Q19** (L05): ____	**Q20** (L05): ____
Q21 (L05): ____	**Q22** (L06): ____	**Q23** (L06): ____	**Q24** (L06): ____	**Q25** (L06): ____
Q26 (L06): ____	**Q27** (L06): ____	**Q28** (L06): ____	**Q29** (L07): ____	**Q30** (L07): ____
Q31 (L07): ____	**Q32** (L07): ____	**Q33** (L07): ____	**Q34** (L07): ____	**Q35** (L07): ____
Q36 (L09a): ____	**Q37** (L09a): ____	**Q38** (L09a): ____	**Q39** (L09a): ____	**Q40** (L09a): ____
Q41 (L09a): ____	**Q42** (L09a): ____	**Q43** (L09b): ____	**Q44** (L09b): ____	**Q45** (L09b): ____
Q46 (L09b): ____	**Q47** (L09b): ____	**Q48** (L09b): ____	**Q49** (L09b): ____	**Q50** (L10): ____
Q51 (L10): ____	**Q52** (L10): ____	**Q53** (L10): ____	**Q54** (L10): ____	**Q55** (L10): ____
Q56 (L10): ____	**Q57** (L11): ____	**Q58** (L11): ____	**Q59** (L11): ____	**Q60** (L11): ____
Q61 (L11): ____	**Q62** (L11): ____	**Q63** (L11): ____	**Q64** (L12): ____	**Q65** (L12): ____
Q66 (L12): ____	**Q67** (L12): ____	**Q68** (L12): ____	**Q69** (L12): ____	**Q70** (L12): ____

L02 — Agents

Q1

rational agent / expected utility

```
A vacuum agent is in a Dirty cell. Its suction motor is unreliable: action Suck leaves
the cell Clean with probability 0.8 and Dirty with probability 0.2. The utility
function assigns U(Clean) = 10 and U(Dirty) = 0. What is the expected utility of Suck,
E[U | Suck]?
```

A) 8

B) 10

C) 2

D) 5

Q2

PEAS classification

A student writes the PEAS for an automated taxi and lists 'GPS' and 'speedometer' under Actuators, alongside 'steering wheel' and 'brake'. Which statement correctly diagnoses this PEAS specification?

A) It is correct, because anything physically attached to the car is an actuator.

B) GPS and speedometer are wrongly placed; they are Sensors (devices that produce percepts), not Actuators.

C) Steering wheel and brake are wrongly placed; they belong under Sensors.

D) GPS belongs under Environment because satellites are external to the agent.

Q3

environment taxonomy (classify scenario)

Consider playing Scrabble against an opponent, where you cannot see the opponent's rack and new tiles are drawn from the bag after every move. Along the six environment dimensions, Scrabble is best classified as:

A) Fully observable, deterministic, episodic, single-agent.

B) Fully observable, stochastic, sequential, single-agent.

C) Partially observable, stochastic, sequential, multi-agent.

D) Partially observable, deterministic, episodic, multi-agent.

Q4 environment taxonomy (deterministic vs stochastic)

A word-jumble solver is given a scrambled word that was selected at random; once shown, the letters never change while you work. By the lecture's convention on where randomness must live, is this environment deterministic or stochastic, and why?

A) Stochastic, because the word was chosen at random.

B) Stochastic, because the solver may guess wrong and have to backtrack.

C) Deterministic, because there is only one agent solving it.

D) Deterministic, because only randomness in state transitions counts, and after the scramble is revealed the transitions are fixed.

Q5 agent hierarchy (simplest architecture)

A factory sorter sits at a conveyor belt. At each instant its sensor fully reports the single item in front of it (its size class), and the correct action depends only on that current reading, never on history. From the five-level hierarchy, what is the SIMPLEST architecture that suffices?

A) Simple reflex agent (condition-action rules on the current percept).

B) Model-based reflex agent, because it needs an internal state estimate.

C) Goal-based agent, because sorting items is a goal.

D) Utility-based agent, because some items are more valuable than others.

Q6 table-driven blow-up arithmetic

```
A table-driven agent stores one action for every possible percept sequence up to its
lifespan T. With |P| = 4 distinct percepts and a lifespan of T = 3 time steps, how
many rows must the table contain?
```

A) 12, from 4*3.

B) 64, from 4^3 only.

C) 84, from 4^1 + 4^2 + 4^3 = 4 + 16 + 64.

D) 20, from 4 + 16.

Q7 learning agent (4 components)

In a learning agent, a self-driving car occasionally tries braking on an unfamiliar road surface even though that is not the locally optimal move, purely to gather informative experience. Which of the four learning-agent components is responsible for this behaviour?

A) The Critic, because it judges the result of the experiment.

B) The Performance element, because it executes the braking action.

C) The Learning element, because it updates the agent's rules.

D) The Problem generator, because it suggests exploratory actions that may not be locally optimal.

L03 — Uninformed Search

Q8 BFS/DFS/UCS/IDS completeness, optimality, time and space (applied)

An agent searches a finite state space with branching factor b = 8. Goals can lie very deep, but the shallowest goal is at depth d, while the maximum path length is m (with m much larger than d). All step costs are equal to 1. The agent's hard constraint is memory: it cannot afford an exponential-sized frontier. Which single statement correctly matches a strategy to its property in THIS scenario?

A) BFS is the safest choice because its space cost O(b^d) is smaller than any other strategy here.

B) DFS is optimal here because all step costs are equal, so the first goal it finds is least-cost.

C) UCS uses only O(b*d) space because the step costs are uniform, so it reduces to a linear-memory search.

D) DFS keeps only one root-to-leaf path plus its siblings, so its space cost is O(b*m) (linear in depth), which is why it survives the memory constraint.

Q9 Frontier data structure (FIFO / LIFO / priority queue)

The four uninformed strategies share one generic algorithm and differ ONLY in how the frontier is ordered. A student must match each strategy to the data structure that produces its expansion order. Which mapping is entirely correct?

A) BFS = FIFO queue; DFS = LIFO stack; UCS = priority queue keyed by g(n); IDS = LIFO stack (one depth-limited DFS at a time).

B) BFS = LIFO stack; DFS = FIFO queue; UCS = priority queue keyed by g(n); IDS = FIFO queue.

C) BFS = priority queue keyed by depth; DFS = LIFO stack; UCS = FIFO queue; IDS = priority queue.

D) BFS = FIFO queue; DFS = priority queue keyed by g(n); UCS = LIFO stack; IDS = FIFO queue.

Q10 Tracing UCS to find an optimal-cost path

Run UCS on this directed graph from S to goal G. Edges (directed) with costs: S->A = 2, S->B = 4, A->C = 4, B->C = 1, C->G = 3, A->D = 7, D->G = 1. UCS tests for the goal when a node is POPPED. What total path cost does UCS return for the optimal solution?

A) 7

B) 8

C) 9

D) 10

Q11 Why UCS goal-tests on POP, not on PUSH

A graph has two goals: S->G1 with cost 6, and S->A (cost 2) then A->G2 (cost 1). S's successors are generated in the order G1 first, then A. A student implements UCS but performs the goal test the moment a node is GENERATED (pushed) instead of when it is popped. What goes wrong, and why does testing on POP fix it?

A) Nothing goes wrong: G1 is generated first at cost 6, the test fires, and 6 is indeed optimal; testing on pop is only a performance optimisation.

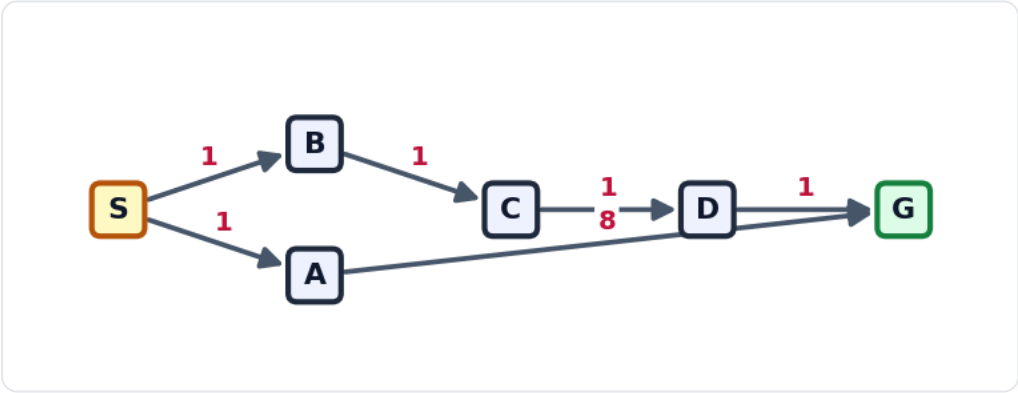
B) Testing on push returns S->G1 at cost 6 immediately, before A is even expanded, so the cheaper S->A->G2 (cost 3) is never found; testing on pop delays the goal test until G2 (g=3) is the cheapest popped node, so the optimum is returned.

C) Testing on push loops forever because G1 keeps being regenerated; testing on pop adds an explored set that stops the loop.

D) Testing on push returns S->A->G2 at cost 3 but reports the wrong cost; testing on pop fixes only the reported cost, not the path.

Q12 BFS vs UCS optimality on a weighted graph

Run BOTH breadth-first search (goal test on expansion, children pushed in edge-list order) and uniform-cost search on the weighted directed graph shown, from start S to goal G. Which pair of returned solution costs is correct?



Weighted directed graph: S to G. The 2-edge route through A is shallow but expensive (8); the 4-edge route through B, C, D is deep but cheap.

A) BFS returns cost 4 and UCS returns cost 9 (BFS is optimal because it finds the shortest path).

B) Both BFS and UCS return cost 4, because both are optimal on any graph.

C) BFS returns cost 9 and UCS returns cost 4 (BFS finds the shallowest goal, not the cheapest; UCS finds the cheapest).

D) BFS returns cost 9 and UCS returns cost 9, because the shallowest path is also the cheapest here.

Q13 IDS time / space tradeoff

A finite tree has branching factor b = 10 and its shallowest goal at depth d = 5. A colleague claims iterative deepening search (IDS) must be far SLOWER than BFS because it re-expands the shallow levels on every pass. Using the lecture's accounting, which statement is correct?

A) IDS has the same big-O time as BFS, O(b^d), because the repeated shallow work is dominated by the deepest level; its advantage over BFS is space, O(b*d) instead of O(b^d).

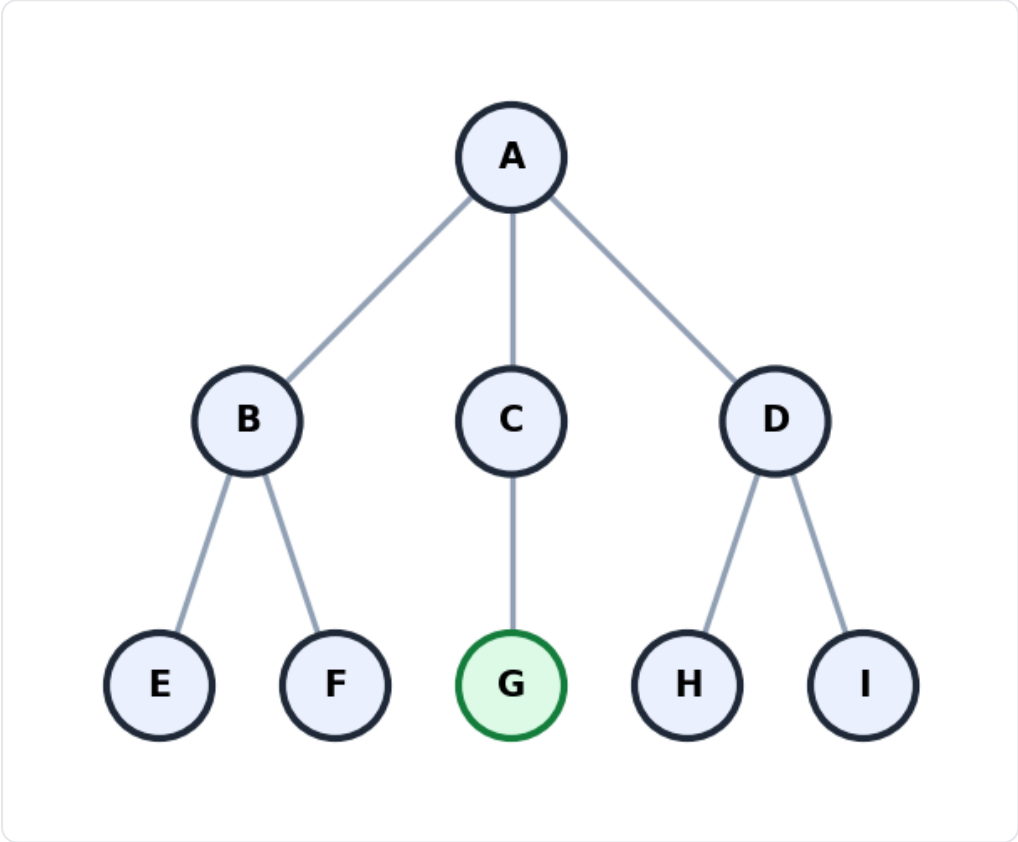
B) IDS is genuinely slower in big-O: it is O(d * b^d), one full BFS per depth level, so the colleague is right.

C) IDS uses the same exponential space O(b^d) as BFS but is faster in time, O(b^(d-1)), because it prunes shallow nodes.

D) IDS is both faster and more memory-hungry than BFS, with time O(b^(d/2)) and space O(b^d).

Q14 DFS expansion order (tree figure)

Depth-first search runs on the tree shown, starting at the root A, with the goal at node G. Children are pushed onto the LIFO stack right-to-left so the LEFTMOST child is expanded first (the lecture's left-first convention). In what order are nodes EXPANDED (popped) until the goal is found?



Search tree for DFS expansion. Root A has children B, C, D (left to right). B has children E, F; C has child G; D has children H, I. Goal = G.

A) A, B, C, D, E, F, G (level-by-level, then the goal)

B) A, B, E, F, C, G

C) A, D, I, H, C, G (rightmost child first)

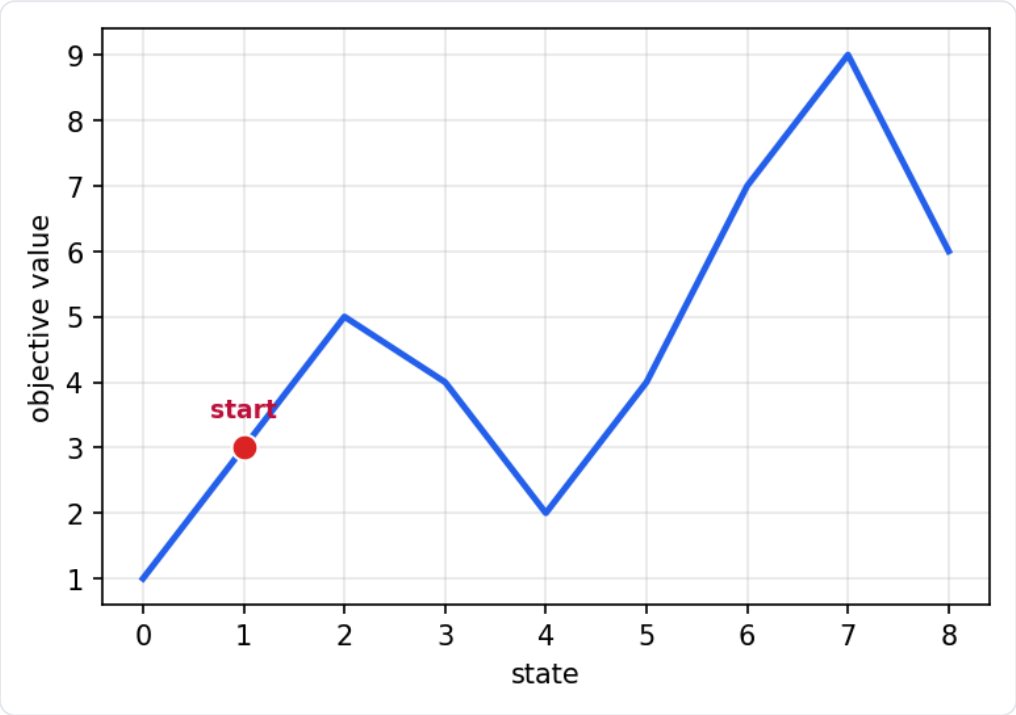
D) A, B, C, E, F, G

L05 — Local Search

Q15

Hill climbing: local vs global maximum

The curve below is a 1-D state-space landscape; the objective value f(s) is to be MAXIMISED. Steepest-ascent hill climbing starts at the marked state and at each step moves to the higher-valued immediate neighbour, stopping when no neighbour strictly improves. Where does it stop, and is that the global maximum?



1-D state-space landscape; objective value to be maximised. Hill climbing starts at the marked state.

A) It stops at state 7 (value 9), which is the global maximum.

B) It stops at state 2 (value 5), a local maximum; it misses the global maximum at state 7 (value 9).

C) It stops at state 0 (value 1) because the left neighbour is missing.

D) It does not stop; it walks back and forth across the whole curve forever.

Q16

Simulated annealing: acceptance probability

A simulated-annealing search MINIMISES an energy/cost E. From a state with E = 20, a random neighbour with E = 24 is proposed at temperature T = 2. Using the acceptance rule P = exp(-DeltaE / T) for a worsening move (where DeltaE = E_next - E_current > 0 is the cost increase), what is the probability this neighbour is accepted?

A) P = exp(-4/2) = exp(-2) ≈ 0.135

B) P = exp(-2/4) = exp(-0.5) ≈ 0.607

C) P = 1, because every proposed move is accepted

D) P = exp(4/2) = exp(2) ≈ 7.39

Q17 Simulated annealing: role of temperature

In a maximisation simulated annealing, a fixed downhill move with Delta = f(next) - f(current) = -3 is repeatedly proposed. Its acceptance probability is exp(Delta/T). Comparing temperature T = 100 against T = 1, which statement is correct?

A) At T = 100, P ≈ 0.05; at T = 1, P ≈ 0.97 — low T explores more aggressively.

B) The probability is exactly 0.5 at both temperatures because Delta is negative.

C) At T = 100, P = exp(-3/100) ≈ 0.97 (almost always accepted); at T = 1, P = exp(-3) ≈ 0.05 (almost never) — high T explores, low T behaves like greedy hill climbing.

D) Both give P = 1 because downhill moves are always accepted regardless of T.

Q18 Local search: constant memory

Compared with breadth-first search on the 8-queens problem (one queen per column, giving 8^8 ≈ 1.68 x 10^7 states), why is plain hill climbing's memory requirement O(1) rather than growing with the state space?

A) Because hill climbing always finds the goal in one step, so it never needs to store more than the start.

B) Because the objective function compresses all 8^8 states into a single number that is stored instead of states.

C) Because hill climbing keeps the entire population of states but each one is only one byte.

D) Because hill climbing stores only the single current state (it discards path information and never keeps a frontier or explored set), so memory does not grow with the number of states.

Q19 Hill climbing on 8-queens: successors and objective

On the 8-queens board (one queen per column, h = number of attacking pairs, MINIMISED), the current board has h = 17. The successor function lets each column's queen move to any of the other 7 rows. How many successors are evaluated per step, and if the best successor has h = 12, what does steepest-ascent hill climbing do?

A) It evaluates 8 successors and stops immediately because 12 < 17.

B) It evaluates 64 successors and stops because no move can reduce conflicts below 17.

C) It evaluates 56 successors and moves to an h = 12 board, since fewer conflicts is better (an improvement of 5).

D) It evaluates 56 successors but stays put, because moving a queen can only increase conflicts.

Q20 Genetic algorithm: roulette-wheel selection

A GA population of 8 chromosomes has fitnesses (1, 2, 3, 1, 3, 5, 1, 2), total F = 18. The cumulative bounds give slices: c1 (0,1], c2 (1,3], c3 (3,6], c4 (6,7], c5 (7,10], c6 (10,15], c7 (15,16], c8 (16,18]. Using 'return the smallest i with F_i >= R', which chromosome does a draw of R = 5 select?

A) Chromosome 2, because its fitness 2 is the closest to 5 minus 3.

B) Chromosome 3, because R = 5 falls in its slice (3, 6].

C) Chromosome 6, because it has the highest fitness and the wheel favours the best.

D) Chromosome 5, because R = 5 equals the chromosome index.

Q21 Genetic algorithm: single-point crossover

Single-point crossover is applied to parents P1 = 1010000000 and P2 = 1001011111 with the cut point after bit position 3 (bits are 1-indexed left-to-right; child1 takes P1's prefix then P2's suffix). What is child1?

A) 1011011111

B) 1001000000

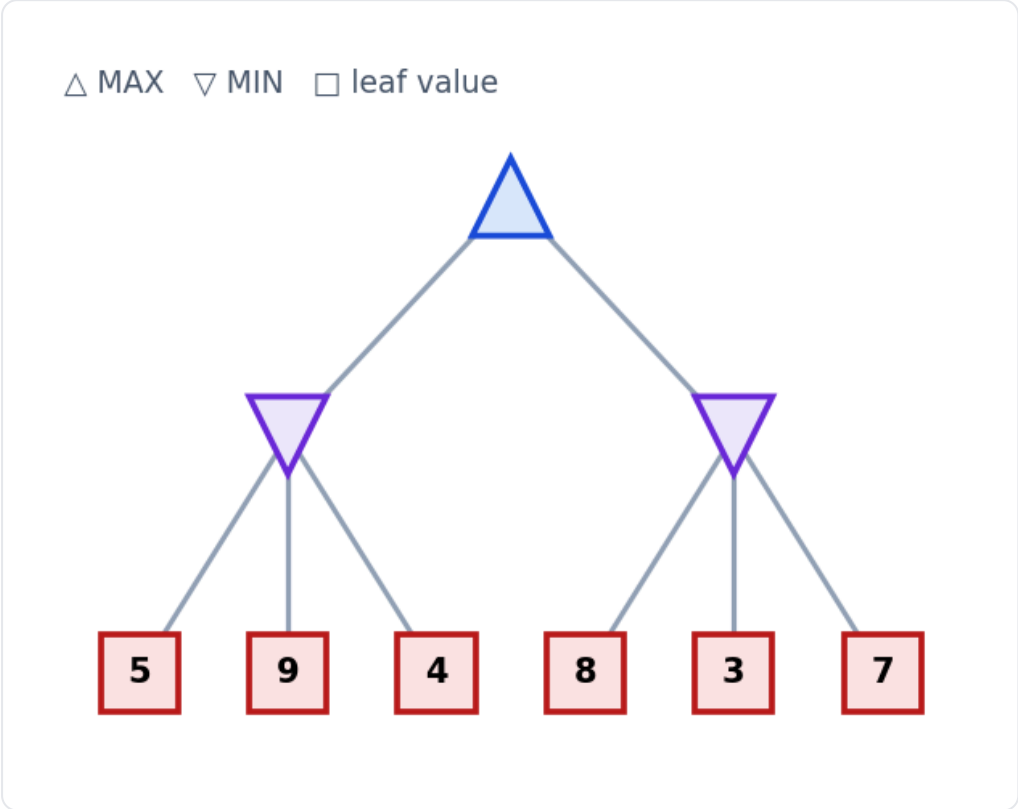
C) 1000000000

D) 1010011111

L06 — Adversarial Search

Q22 Minimax value at the root

In the game tree below, MAX moves at the root and each of its two children is a MIN node. Reading the leaves left-to-right, what is the minimax value backed up to the root?



MAX root with two MIN children; leaves read left-to-right.

A) 9 (the largest leaf in the tree)

B) 3 (the smallest of the two MIN-node values)

C) 4

D) 5 (the average of the two MIN-node values)

Q23 Minimax decision (best first move)

For the same tree as the previous question (left MIN = 4, right MIN = 3, root value = 4), which move should MAX make at the root, and why?

A) Play the LEFT branch, because it has the higher backed-up MIN value (4 > 3), so it gives the best guaranteed worst-case payoff.

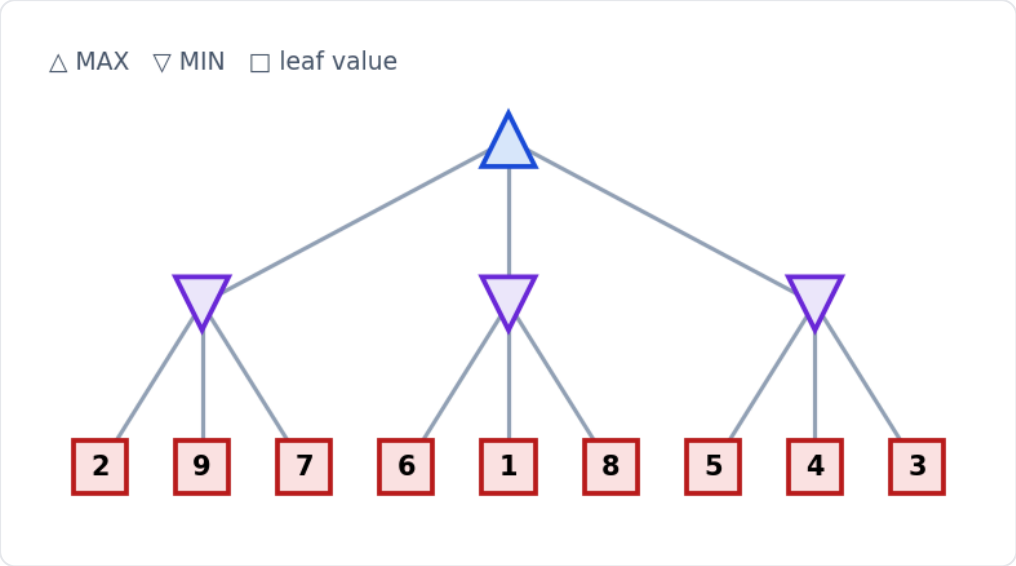
B) Play the RIGHT branch, because its subtree contains a smaller worst case that MIN will be forced into.

C) Play the LEFT branch, because its subtree contains the single largest leaf (9).

D) Either branch is equally good, because the root value is 4 regardless of which child MAX picks.

Q24 Alpha-beta pruning (which leaves are pruned)

Run alpha-beta on the tree below with children expanded strictly LEFT-TO-RIGHT (DFS). The root is MAX with three MIN children; leaf values are shown. Which leaf (or leaves) are PRUNED, i.e. never evaluated?



MAX root, three MIN children, expanded left-to-right; leaves read left-to-right.

A) No leaves are pruned; alpha-beta must still examine all nine leaves on this tree.

B) Only the leaf with value 8 (the third child of the middle MIN).

C) The leaves 4 and 3 (the last two children of the right MIN).

D) The leaves 1 and 8 (the last two children of the middle MIN).

Q25 Effect of move ordering on pruning

Take a fixed game tree on which alpha-beta with one particular left-to-right ordering prunes several subtrees. If you now REVERSE the order in which every node expands its children (worst-for-the-mover first at every node), what happens?

A) The minimax value at the root changes, because pruning a different set of subtrees changes the backed-up answer.

B) Alpha-beta becomes incomplete and may fail to return a value for the root.

C) Pruning improves, because reversing the order always expands the best child first.

D) The root's minimax value is unchanged, but more nodes may now be expanded; in the worst ordering alpha-beta prunes nothing and degenerates to plain minimax (O(b^d)).

Q26 Evaluation function at a depth cutoff

A depth-limited alpha-beta search reaches its horizon (depth = 0) at a board position where the game is NOT yet over. What value should the search back up from that node, and what is its status?

A) The rule-given utility U(s), because every node the search stops at is treated as a terminal.

B) Zero, because a non-terminal position has no defined value until the game ends.

C) The evaluation function Eval(s), a heuristic ESTIMATE of MAX's utility; it may be wrong, so the search is not guaranteed optimal.

D) The exact Minimax(s) value, obtained by continuing the recursion past the horizon until real terminals are reached.

Q27 Minimax assumption / sub-optimal opponent

A minimax agent computes a root value of 5 against an opponent assumed to play optimally. During the real game the opponent blunders, choosing a branch whose minimax value is 8 (better for MAX) instead of the optimal-for-MIN branch worth 5. What can you conclude about MAX's realised payoff?

A) MAX's realised payoff can only be at least the minimax value (>= 5); against a sub-optimal opponent it may end up higher, never lower.

B) MAX's realised payoff is exactly 5, because the precomputed minimax value is fixed regardless of how the opponent actually plays.

C) MAX's realised payoff drops below 5, because the agent planned for a different opponent than the one it faced.

D) The result is undefined, because minimax only works when the opponent plays optimally.

Q28 Properties: time/space complexity and ply

A two-player game has branching factor b = 4 and you search it fully to a depth of d = 6 ply with plain minimax (depth-first). Which statement about the search is correct?

A) Time is O(b * d) = 24 node expansions, because DFS only stores one path at a time.

B) Space is O(b^d) = 4^6 nodes, because minimax must keep the entire tree in memory to back values up.

C) Switching to alpha-beta would change the backed-up root value but keep the same O(b^d) worst-case time.

D) Worst-case time is O(b^d) = 4^6 expansions and space is O(b*d) = O(24); one ply is a single player's half-move, so d = 6 means six half-moves (three by each side).

L07 — Constraint Satisfaction (CSP)

Q29

Formulating a problem as a CSP

You must formulate 4-queens as a CSP using the compact formulation from the lecture (one variable per row, baking in the one-queen-per-row rule). Which choice correctly states the variables, domains, and the pairwise constraint between two rows i and j?

A) Variables X1..X4 (one per row), each domain {1,2,3,4} (the queen's column); for i != j require Xi != Xj AND |Xi - Xj| != |i - j|.

B) Variables X1..X4 (one per row), each domain {0,1}; for i != j require Xi != Xj only.

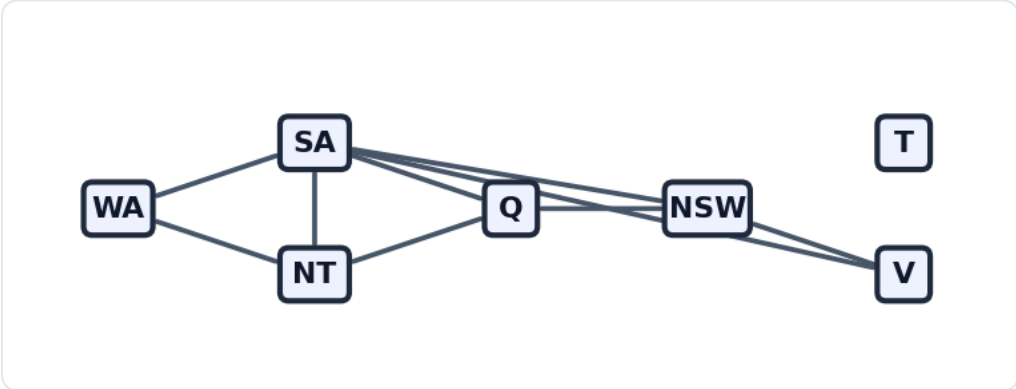
C) One Boolean variable per cell (16 variables), each domain {1,2,3,4}; constrain at most one 1 per row, column, and diagonal.

D) Variables X1..X4 (one per column), each domain {1,2,3,4} (the queen's row); for i != j require only |Xi - Xj| != |i - j|.

Q30

Constraint graph + degree heuristic

The graph shows the Australia map-coloring CSP (each edge is a 'must differ' constraint; every variable's domain is {red, green, blue}). No variable has been assigned yet. Which variable does the degree heuristic select first, and why?



Australia map-coloring constraint graph; each edge is a must-differ constraint, domain {red, green, blue}.

A) T, because as an isolated node it can take any value without risk.

B) SA, because it participates in the most constraints (5 edges) on still-unassigned variables.

C) WA, because it is drawn first and borders both NT and SA.

D) NT, because it lies inside the WA-NT-SA triangle and so is maximally constrained.

Q31 Forward checking (which values get eliminated from a neighbor)

Using the same Australia constraint graph (domains {red, green, blue}), backtracking with forward checking has just assigned WA = red and then Q = green. No other variable is assigned. After forward checking propagates these two assignments, what is SA's remaining domain?

A) {red, green, blue} -- SA is unaffected because it has not been assigned.

B) {red, blue} -- only Q = green removes a value from SA.

C) {blue} -- WA = red removes red and Q = green removes green from SA.

D) {} -- SA is already wiped out, so the solver backtracks immediately.

Q32 Forward checking on 4-queens

In 4-queens (compact formulation, X1..X4 = queen's column in each row, domain {1,2,3,4}), the solver assigns X1 = 1 (queen at row 1, column 1) and runs forward checking. What is the resulting domain of X3 (row 3)?

A) {1, 3} -- X3 keeps the column of X1 and the cell on X1's diagonal.

B) {2, 4} -- X3 loses column 1 (same column) and column 3 (on X1's diagonal).

C) {2, 3, 4} -- only the shared column is removed.

D) {} -- row 3 has no legal column, so the solver backtracks.

Q33 Arc consistency / AC-3 (is an arc consistent? what gets pruned?)

Two variables x and y have domains Dx = {1,2,3} and Dy = {1,2,3} with the single constraint x < y. Enforcing arc consistency, you first REVISE the directed arc x -> y. Which statement is correct?

A) The arc x -> y is already consistent; nothing is removed from Dx.

B) REVISE removes 1 from Dx, because no y satisfies 1 < y.

C) REVISE removes 3 from Dx, because no y in {1,2,3} satisfies 3 < y, giving Dx = {1,2}.

D) REVISE removes 1 from Dy, because the arc x -> y prunes the head variable y.

Q34 Variable ordering heuristics (MRV with degree tie-break)

Mid-search in a map-coloring CSP the current legal domains are: V1 = {red, green, blue}, V2 = {green, blue}, V3 = {red}, V4 = {green, blue}. V3 is already assigned (red). Among V2 and V4 (which tie on remaining-domain size), V2 has 3 unassigned neighbours and V4 has 1. Which variable does backtracking choose next under MRV with the degree heuristic as tie-breaker?

A) V1, because it has the most colours available and is least likely to fail.

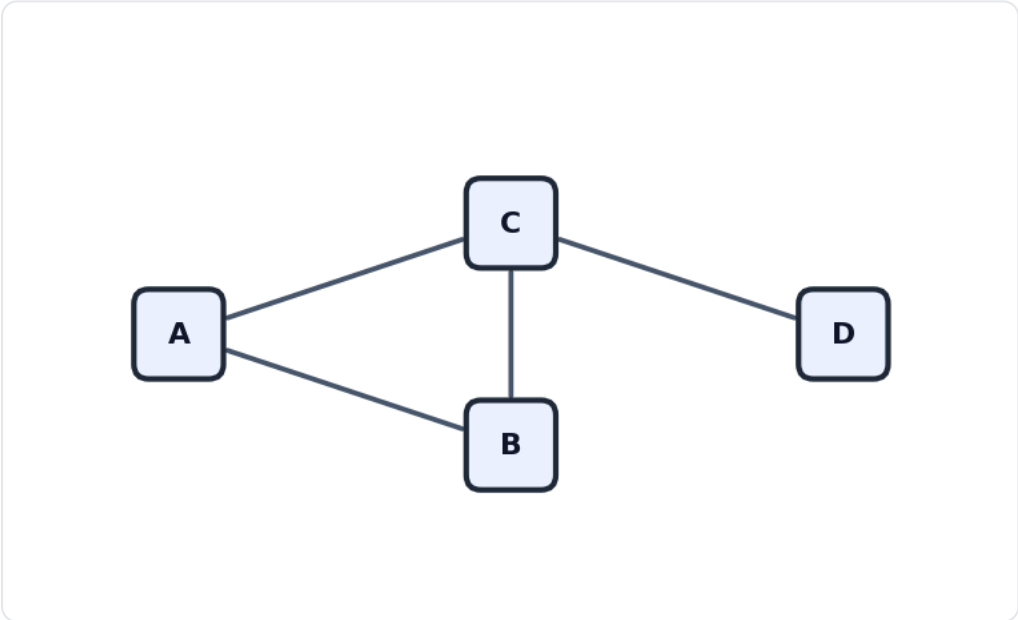
B) V3, because a domain of size 1 is the absolute minimum-remaining-values choice.

C) V4, because the degree heuristic prefers the variable with the fewest unassigned neighbours.

D) V2, because after MRV ties it and V4 at two values, the degree heuristic prefers V2's 3 unassigned neighbours.

Q35 Arc consistency vs forward checking (map-coloring)

The graph shows a 4-region map-coloring CSP (edges are must-differ constraints, every domain {red, green, blue}). The solver has assigned A = red, then propagated. The current domains are A = {red}, B = {green, blue}, C = {green, blue}, D = {red, green, blue}. Forward checking now reports no failure. What does enforcing full arc consistency reveal that forward checking missed, if anything?



4-region map-coloring CSP; triangle A-B-C plus pendant D, each edge a must-differ constraint, domain {red, green, blue}.

A) Nothing extra; B and C each still have two colours, so the state is already arc-consistent.

B) It prunes D to {} because D borders C, signalling failure.

C) Forward checking was wrong to keep B = {green, blue}; arc consistency restores red to B.

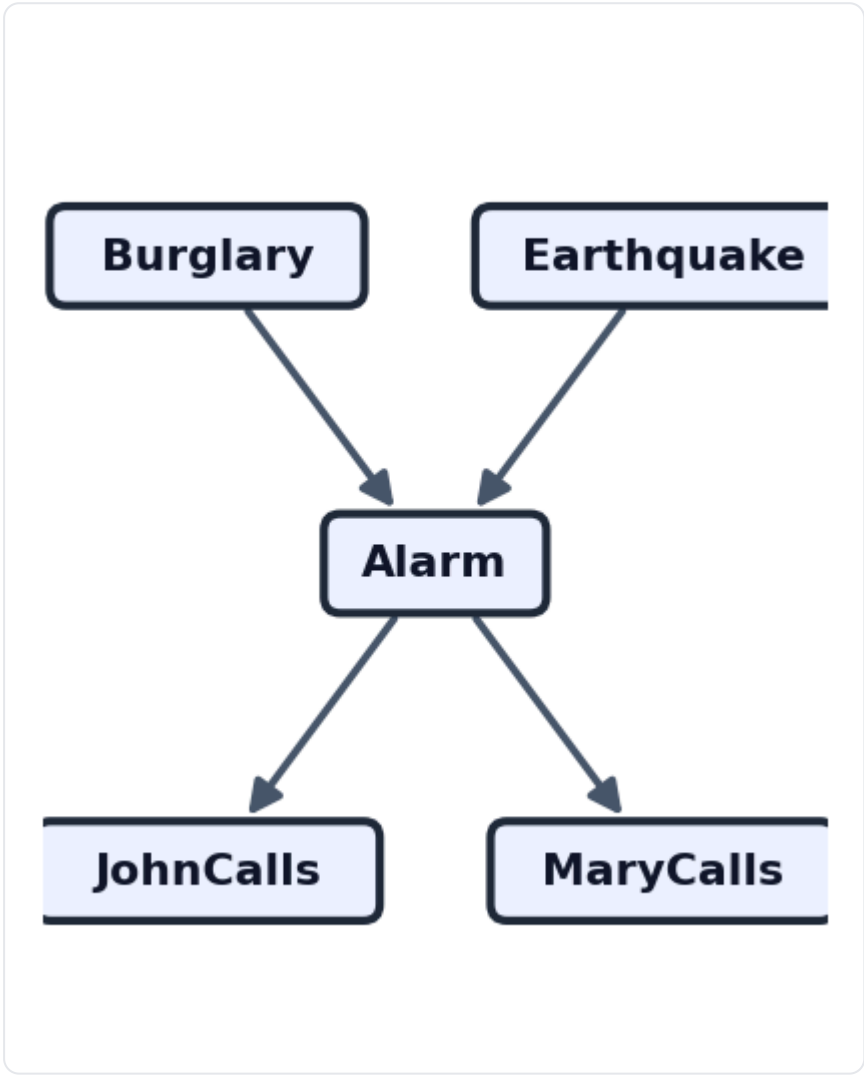
D) Both arcs B -> C and C -> B are consistent here (each value of B has a different-coloured support in C and vice versa), so arc consistency also finds no failure -- it only beats FC when a shared neighbour is forced to a single value.

L09a — Bayesian Networks

Q36

joint factorization from a DAG

Use the Bayesian network in the figure (the classic alarm network): Burglary (B) and Earthquake (E) are both parents of Alarm (A), and Alarm is the sole parent of both JohnCalls (J) and MaryCalls (M). Reading the BN factorization P(X1,...,Xn) = product of P(Xi | Parents(Xi)) directly off this structure, the full joint P(B,E,A,J,M) equals which product?



Alarm Bayesian network: Burglary and Earthquake cause Alarm; Alarm triggers JohnCalls and MaryCalls.

A) P(B)*P(E)*P(A)*P(J)*P(M)

B) P(B|A)*P(E|A)*P(A|J,M)*P(J)*P(M)

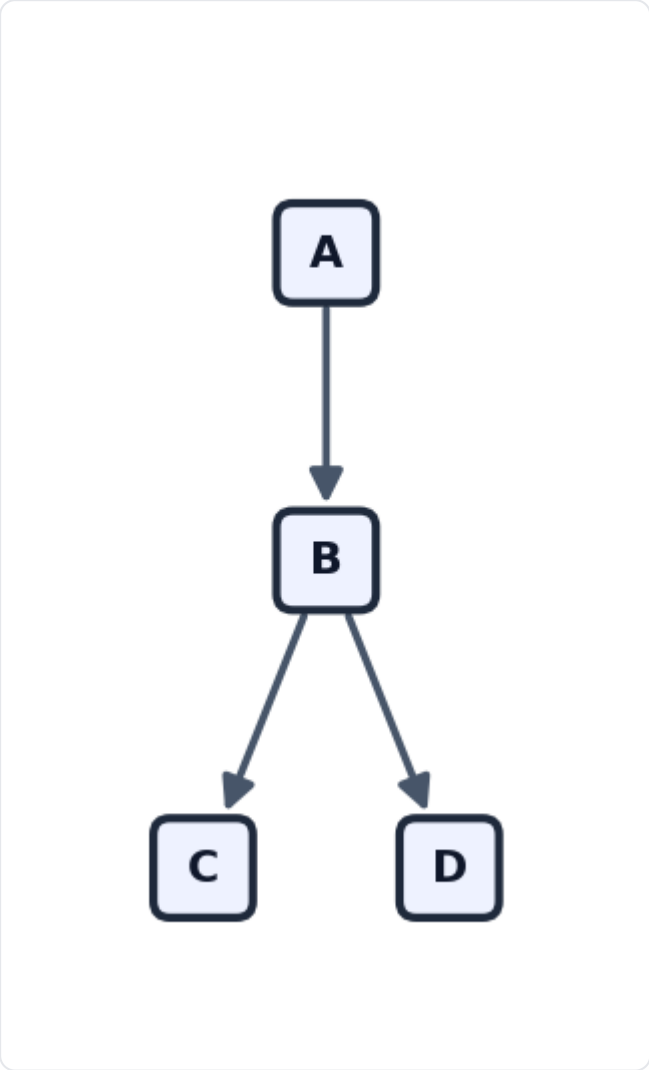
C) P(B)*P(E)*P(A|B,E)*P(J|A)*P(M|A)

D) P(B)*P(E)*P(A|B,E)*P(J|A,M)*P(M|A,J)

Q37

computing a joint entry from CPTs

Consider the Bayesian network in the figure: A -> B, and B -> C, B -> D (so A is the root, B has parent A, and C and D each have the single parent B). The relevant CPT entries are: P(A=true)=0.4 ; P(B=true | A=true)=0.3 ; P(C=true | B=true)=0.1 ; P(D=true | B=true)=0.95. What is the joint probability P(A=true, B=true, C=true, D=true)?



A -> B -> {C, D}: A is the root, B depends on A, and C and D each depend only on B.

A) 0.0114

B) 0.114

C) 0.012

D) 0.0285

Q38 computing a joint entry (alarm network)

Using the alarm network of Q1 with CPTs: P(+b)=0.001 ; P(+e)=0.002 ; P(+a|+b,+e)=0.95, P(+a|+b,~e)=0.94, P(+a|~b,+e)=0.29, P(+a|~b,~e)=0.001 ; P(+j|+a)=0.9, P(+j|~a)=0.05 ; P(+m|+a)=0.7, P(+m|~a)=0.01. Compute the probability of the atomic event 'burglary, no earthquake, alarm sounds, John does NOT call, Mary calls' = P(+b, ~e, +a, ~j, +m). (Note: ~ denotes false.)

A) about 5.9 x 10^-4

B) about 6.6 x 10^-5

C) about 5.9 x 10^-3

D) about 6.3 x 10^-4

Q39 conditional independence / Markov condition

In the lecture-late Bayesian network the edges are M -> R, M -> L, S -> L, L -> T (M and S are independent roots; R has parent M only; L has parents M and S; T has parent L only). By the Markov condition, a node is conditionally independent of all its non-descendants given its parents. Given that we know the value of M, which set of variables is the topic node R conditionally independent of?

A) R is independent of M, S, and L (but not T) given M.

B) R is independent of nothing, because R is correlated with T through the network.

C) R is independent only of S given M, since L and T are downstream of M.

D) R is conditionally independent of S, L, and T given M.

Q40 counting independent parameters

For the alarm network of Q1 (5 Boolean variables, with parent sets: B = none, E = none, A = {B,E}, J = {A}, M = {A}), how many INDEPENDENT probability parameters are needed to fully specify all the CPTs? (A Boolean node with k Boolean parents needs 2^k independent parameters.)

A) 20

B) 31

C) 10

D) 32

Q41

collider / explaining away

In the alarm network of Q1, Burglary (B) and Earthquake (E) are the two parents of Alarm (A), with no edge between B and E. The CPTs give P(+b)=0.001 and, after computing, P(+b | +a) ~ 0.37 while P(+b | +a, +e) ~ 0.003. Which statement correctly describes the relationship between Burglary and Earthquake?

A) B and E are dependent marginally but become independent once Alarm is observed, because conditioning always removes dependence.

B) B and E are independent marginally (P(B|E)=P(B)), but become dependent once Alarm is observed; observing the earthquake explains away the alarm and lowers P(burglary).

C) B and E are independent both marginally and after observing Alarm, since there is no edge between them.

D) B and E are dependent both marginally and conditionally, because they share the common child Alarm.

Q42

inference by enumeration

Using the alarm network of Q1 with P(+b)=0.001, P(+e)=0.002, and P(+a|+b,+e)=0.95, P(+a|+b,~e)=0.94, P(+a|~b,+e)=0.29, P(+a|~b,~e)=0.001, compute the marginal probability that the alarm sounds, P(A=true), by inference by enumeration (sum P(+a|b,e)*P(b)*P(e) over all four combinations of B and E).

A) about 0.50

B) about 0.94

C) about 0.045

D) about 0.0025

L09b — Hidden Markov Models

Q43 Markov assumption / conditional independence structure

An HMM is used to tag a sentence: words are observed, part-of-speech tags are the hidden states. Under the HMM's two structural assumptions, which factorization of the joint probability P(O, Q) of a word sequence O = o1..oT and tag sequence Q = q1..qT is correct?

A) P(O,Q) = product over t of P(o_t | o_1..o_{t-1}) * P(q_t | q_1..q_{t-1}) -- each factor conditions on the full history

B) P(O,Q) = pi_{q1} * b_{q1}(o1) * product from t=2 to T of a_{q_{t-1} q_t} * b_{q_t}(o_t) -- transitions depend only on the previous state, emissions only on the current state

C) P(O,Q) = product over t of P(o_t | q_t) only -- the tags form an independent set, so transitions do not appear

D) P(O,Q) = pi_{q1} * product from t=2 to T of a_{q_{t-1} q_t} -- emissions are absorbed into the transition matrix

Q44 Identifying transition vs emission vs initial (prior) probabilities

In the ice-cream HMM the lecture states: 'On the first day, the weather is HOT with probability 0.8 and COLD with probability 0.2; if today is HOT, tomorrow is HOT with probability 0.7 and COLD with probability 0.3; on a HOT day the person eats 3 ice creams with probability 0.4.' Classify the three bolded numbers 0.8, 0.7, and 0.4 in order.

A) 0.8 = transition a_{HH}, 0.7 = emission b_H(3), 0.4 = initial pi_H

B) 0.8 = emission b_H(1), 0.7 = initial pi_H, 0.4 = transition a_{HH}

C) 0.8 = initial pi_H, 0.7 = transition a_{HH}, 0.4 = emission b_H(3)

D) 0.8 = initial pi_H, 0.7 = emission b_H(3), 0.4 = transition a_{HH}

Q45 One step of filtering / forward algorithm + normalization (2-state HMM)

```
A 2-state HMM has hidden states Rain and Sun with initial distribution pi = (Rain 0.5, Sun 0.5). On day 1 you observe 'umbrella'. The emission probabilities for observing an umbrella are b_Rain(umbrella) = 0.9 and b_Sun(umbrella) = 0.2. Using the forward initialization alpha_1(j) = pi_j * b_j(o1) and then normalizing, what is the filtered belief P(q1 | umbrella)?
```

A) alpha = <0.5*0.9, 0.5*0.2> = <0.45, 0.10>, normalize (sum 0.55) -> <Rain 0.818, Sun 0.182>

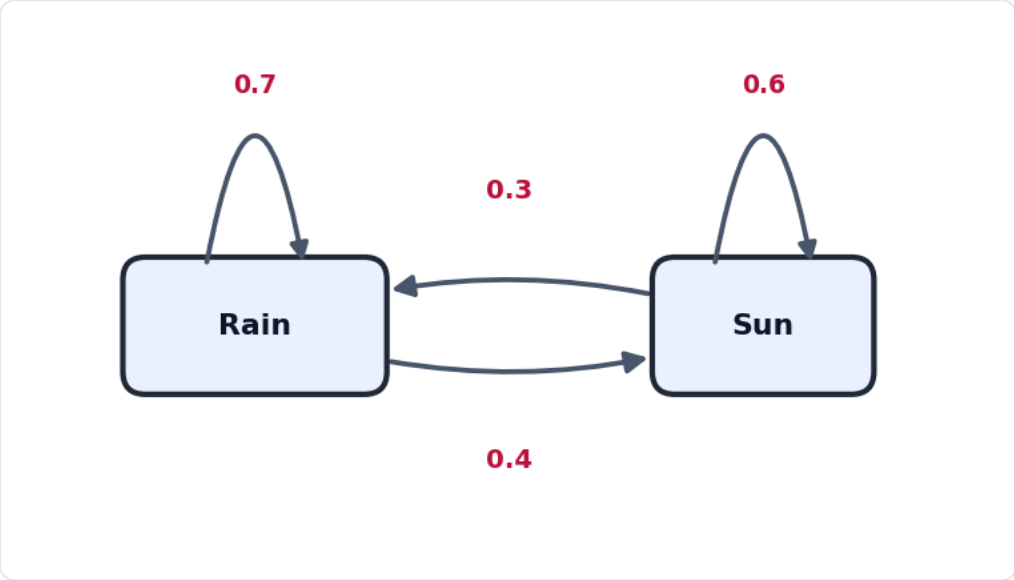
B) alpha = <0.5*0.9, 0.5*0.2> = <0.45, 0.10>, no normalization needed -> <Rain 0.45, Sun 0.10>

C) alpha = <0.5+0.9, 0.5+0.2> = <1.4, 0.7>, normalize -> <Rain 0.667, Sun 0.333>

D) alpha = <0.9, 0.2>, normalize (sum 1.1) -> <Rain 0.818, Sun 0.182>

Q46 Transition model / one-step prediction using the transition diagram

Use the 2-state weather HMM in the figure (transition probabilities labelled on the edges). Today you are certain it is Rain, so the current belief is P(today) = (Rain 1.0, Sun 0.0). With no observation tomorrow, you propagate one step forward through the transition matrix only. What is P(tomorrow = Rain)?



Two-state weather HMM transition diagram. Nodes are hidden states; directed edges (including self-loops) are labelled with transition probabilities. Each state's outgoing probabilities sum to 1.

A) 0.3 -- read the Rain->Sun edge

B) 0.5 -- a 2-state chain is symmetric so each state gets half

C) 0.42 -- multiply the two self-loops 0.7 * 0.6

D) 0.7 -- sum over today's states of P(today=i) * a_{i,Rain} = 1.0*0.7 + 0.0*0.4

Q47 Forward algorithm recursion (one numeric step, ice-cream HMM)

Ice-cream HMM (trellis convention): pi = (COLD 0.2, HOT 0.8); transitions a_CC=0.6, a_CH=0.4, a_HC=0.3, a_HH=0.7; emissions b_C=(0.5,0.4,0.1) and b_H=(0.2,0.4,0.4) over observations {1,2,3}. For O = 3,1,3 the day-1 forward values are alpha_1(HOT)=0.32 and alpha_1(COLD)=0.02. Apply the recursion alpha_t(j) = [sum_i alpha_{t-1}(i) a_{ij}] * b_j(o_t) to compute alpha_2(HOT), where o2 = 1.

A) alpha_2(HOT) = [0.32*0.7 + 0.02*0.4] * 0.2 = [0.224 + 0.008]*0.2 = 0.232*0.2 = 0.0464

B) alpha_2(HOT) = [0.32*0.7 + 0.02*0.4] = 0.224 + 0.008 = 0.232 (emission applied later)

C) alpha_2(HOT) = [0.32*0.3 + 0.02*0.6] * 0.2 = [0.096 + 0.012]*0.2 = 0.0216

D) alpha_2(HOT) = max(0.32*0.7, 0.02*0.4) * 0.2 = 0.224*0.2 = 0.0448

Q48 Normalization of a belief vector

```
After running the forward initialization on day 1 of the ice-cream HMM (O starts with
a 3), the unnormalized forward column is alpha_1 = <HOT 0.32, COLD 0.02>. These are
joint probabilities P(o1, q1). What is the normalized filtered belief P(q1 | o1) over
(HOT, COLD)?
```

A) <HOT 0.32, COLD 0.02> -- already a valid distribution, no normalization needed

B) <HOT 0.640, COLD 0.040> -- divide each by the larger value 0.5

C) <HOT 0.941, COLD 0.059> -- divide each by the column sum 0.34

D) <HOT 0.059, COLD 0.941> -- divide each by the column sum 0.34 then swap to match state order

Q49 Most-likely-state-sequence (Viterbi) vs forward likelihood -- tiny numeric case

For a tiny HMM the only two possible hidden-state sequences that could have produced an observation O have joint probabilities P(O, Q_a) = 0.012 and P(O, Q_b) = 0.003. Both forward and Viterbi process this same trellis. What do the forward algorithm and the Viterbi algorithm respectively output for this O?

A) Forward outputs 0.012 (the max joint); Viterbi outputs 0.015 (the sum)

B) Forward outputs 0.015 (the sum of the joints = P(O)); Viterbi outputs 0.012 (the max joint, the best single path)

C) Both output 0.015, because both algorithms total over all state sequences

D) Both output 0.012, because both algorithms keep only the single most likely path

L10 — Intro to Machine Learning

Q50 Supervised vs unsupervised vs reinforcement learning

A robot vacuum is dropped into a brand-new house with no map and no labelled training data. It drives around, and each time it cleans a previously dirty patch it gets +1, while each time it bumps a wall it gets -1. Over many runs it learns which movement choices in which situations raise its long-run score. Which branch of machine learning does this match, per L10's definitions?

A) Supervised learning, because the +1 / -1 signals are output labels for each input state

B) Unsupervised learning, because there is no labelled training set and the robot finds structure on its own

C) Reinforcement learning, because an agent interacts with an environment of states, actions and rewards and gathers its own data to maximise long-term reward

D) Semi-supervised learning, because only some of the states carry a reward

Q51 Supervised vs unsupervised learning (classify a task)

A streaming service has a large table of users' raw listening features (genres played, time of day, session length) with NO category labels attached. It wants to automatically discover groups of similar listeners so marketing can later inspect the groups. Using L10's framework, this task is best described as:

A) Unsupervised learning (clustering) — inputs only, no labels, discover structure

B) Supervised classification — assign each user to one of a finite set of known marketing segments

C) Supervised regression — predict a continuous loyalty score per user

D) Reinforcement learning — the marketing team rewards good groupings over time

Q52 Classification vs regression (classify a task)

A weather company builds a model that eats today's temperature, humidity, pressure and wind and outputs a single real-valued number: tomorrow's noon temperature in degrees Celsius (e.g. -3.4, 18.7, 27.1). Per L10 §3.2, this supervised task is:

A) Binary classification, because tomorrow is either warmer or colder than today

B) Multi-class classification, because each whole-degree value is a separate class

C) Clustering, because temperatures naturally group into seasons

D) Regression, because the output is a continuous real value rather than a discrete class

Q53 Overfitting vs underfitting (diagnose from error pattern)

You grow a decision tree to full depth on a small dataset. You measure: training accuracy = 100% (training error 0%), test accuracy = 62% (test error 38%). As you grew the tree deeper, training error kept dropping while test error first fell and then climbed back up. Per L10 §6.1, what is happening and what is the fix?

A) Underfitting — the model is too simple; grow the tree deeper / add capacity

B) Overfitting — the model is more complex than necessary; prune the tree (pre- or post-pruning)

C) The model is well-fit — 100% training accuracy means it has learned the task perfectly

D) A train/test leak — the only explanation for any gap between training and test accuracy

Q54 Computing accuracy / precision / recall from a confusion matrix

```
A spam classifier is evaluated on 200 test emails. Class 'Spam' is the positive class.
The confusion matrix is:

                  Predicted Spam   Predicted Not-Spam
  Actual Spam            40                20
  Actual Not-Spam        10               130

Using precision = TP/(TP+FP) and recall = TP/(TP+FN), what are the precision and
recall for the Spam class?
```

A) precision = 40/(40+20) = 0.667 and recall = 40/(40+10) = 0.80

B) precision = 40/(40+10) = 0.80 and recall = 40/(40+20) = 0.667

C) precision = 130/(130+10) = 0.929 and recall = 130/(130+20) = 0.867

D) precision = (40+130)/200 = 0.85 and recall = 40/200 = 0.20

Q55 Train/validation/test split and cross-validation purpose

While tuning a DecisionTreeClassifier you must pick the best value of max_depth. A teammate proposes: 'Train each candidate depth on the training set, then pick the depth that gives the highest accuracy on the test set, and report that same test accuracy as the model's final score.' Per L10's train/test logic (§3.3, §6.1), what is the flaw and the correct practice?

A) No flaw — the test set is exactly what you should use to choose hyperparameters and to report the score

B) Tune max_depth on a separate validation set (or via cross-validation), and keep the test set untouched for one final unbiased estimate

C) Tune max_depth on the training accuracy instead, since training error is the reliable estimate of generalisation

D) Skip the split entirely — fit on all data and report training accuracy, which avoids any leakage

Q56 Bias-variance idea (ensembles, high-variance trees)

On the cheat dataset, swapping just a couple of training rows changes which attribute wins at the root, producing a noticeably different single decision tree each time (Figure 5). Per L10 (§4.9, §6.4), this sensitivity is best described as high variance, and the standard remedy is to:

A) Train many trees on bootstrap samples and combine them (bagging / random forest) so the individual trees' variance largely cancels in the vote

B) Grow each single tree to greater depth, since a deeper tree is less sensitive to changes in the training data

C) Switch the splitting criterion from Gini to entropy, which removes variance because the two measures usually pick the same split

D) Train on the test set as well as the training set, so the tree sees every row and stops changing

L11 — Regression

Q57 prediction from a linear model

A fitted simple regression model is Sales = 51.849 + 7.527 * Advertising, where Advertising is in thousands of dollars and Sales is in thousands of units. A region spends $12,000 on advertising (Advertising = 12). What does the model predict for Sales (in thousands of units)?

A) 63.376

B) 142.173

C) 90.324

D) 59.376

Q58 MSE / SSE on a tiny dataset

A model predicts yhat = 1 + 2*x. The dataset has three points: (x=1, y=4), (x=2, y=4), (x=3, y=8). Using MSE = (1/n)*sum((y_i - yhat_i)^2), what is the MSE?

A) 3

B) 0

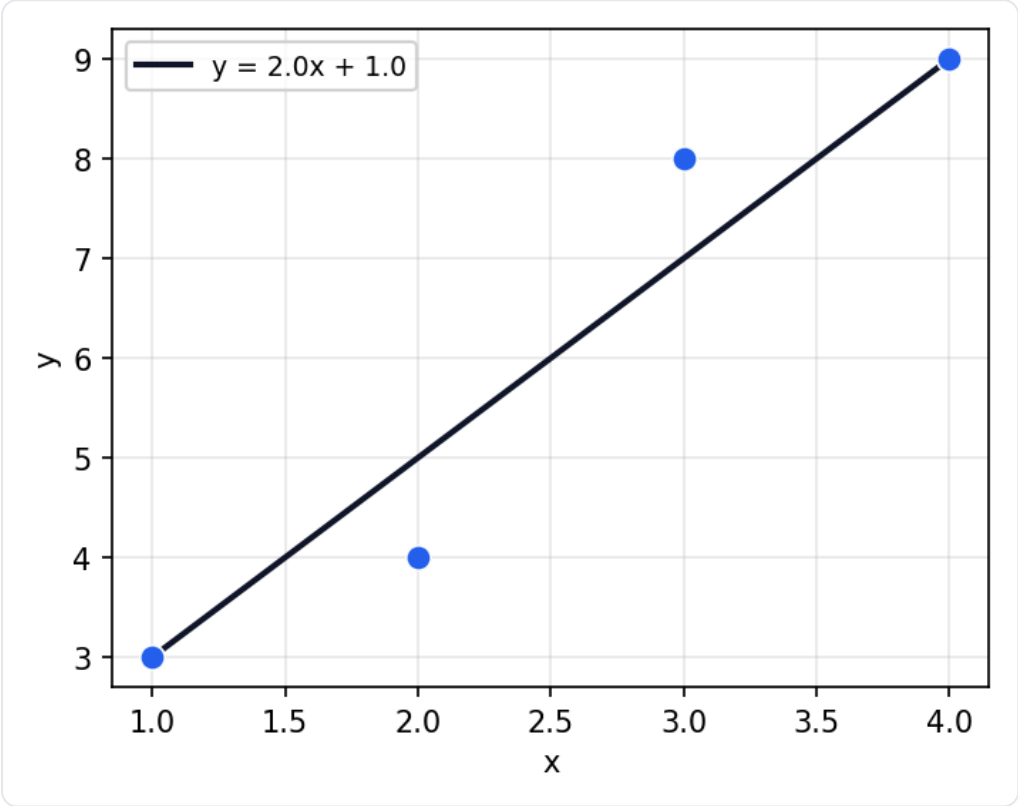
C) 1

D) 9

Q59

residuals from a fitted line (figure)

The scatter shows four data points and the fitted line yhat = 2*x + 1. Reading off the highlighted point at x = 3 (observed y = 8), what is its residual r = y - yhat, and is the point above or below the line?



*Four points with fitted line yhat = 2*x + 1; the point at x=3 is observed at y=8.*

A) r = +1, above the line

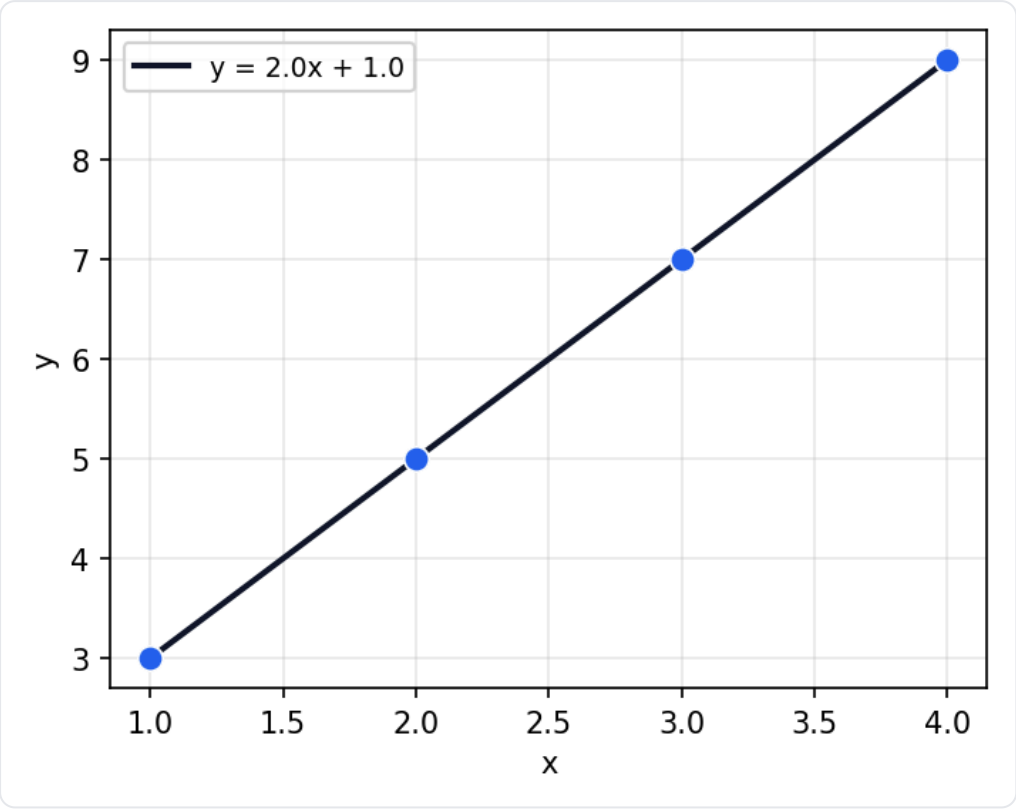
B) r = -1, below the line

C) r = +7, above the line

D) r = 0, exactly on the line

Q60 R-squared interpretation (figure)

The scatter shows four points that all lie exactly on the fitted line yhat = 2*x + 1, so every residual is zero. What is the coefficient of determination R^2 for this training-set fit, and what does it mean?



*Four points lying exactly on the line yhat = 2*x + 1 (perfect fit, all residuals zero).*

A) R^2 = 0, the model explains none of the variability

B) R^2 = 0.5, the model explains half of the variability

C) R^2 = 1, the model explains all of the variability (SSE = 0)

D) R^2 cannot be computed because the slope is not zero

Q61 gradient descent — learning rate effect

ML Lab 2 fits the same OLS line by gradient descent, iteratively stepping the coefficients downhill on the SSE/MSE surface. A student sets the learning rate far too large and observes the loss bouncing up and down and growing instead of settling. What is the most likely cause, and the correct fix?

A) The learning rate is too large, causing the updates to overshoot the minimum and diverge; reduce the learning rate.

B) The learning rate is too small, so it cannot reach the minimum; increase it further.

C) The SSE surface has many local minima, so no learning rate can work; switch to a different loss.

D) Gradient descent cannot fit a linear model at all; only the normal equation works.

Q62 normal equation vs iterative gradient descent

For ordinary least squares with one or a few predictors, the lecture solves the fit in closed form (the normal equations) while ML Lab 2 also uses iterative gradient descent. Because the SSE objective for linear regression is convex, which statement best compares the two approaches?

A) They minimise different objectives, so they generally give different coefficients.

B) Both minimise SSE and, at convergence, gradient descent reaches the same coefficients as the closed-form normal-equation solution.

C) Gradient descent has no hyperparameters, whereas the normal equation requires a learning rate.

D) The normal equation can land in a local minimum, so gradient descent is always more accurate.

Q63 overfitting with polynomial regression

Using polynomial features (adding x^2, x^3, ... as columns), a student fits a degree-10 polynomial to just 12 noisy training points and gets training R^2 = 0.999, but the curve wiggles wildly between points and performs poorly on new test data. What is the best description of what happened, and the appropriate remedy?

A) Underfitting; increase the polynomial degree further to capture more structure.

B) Multicollinearity caused by too few points; add more predictors of a different kind.

C) Overfitting; the high-degree model memorised training noise, so use a lower degree (or more data / regularisation) and judge on held-out test data.

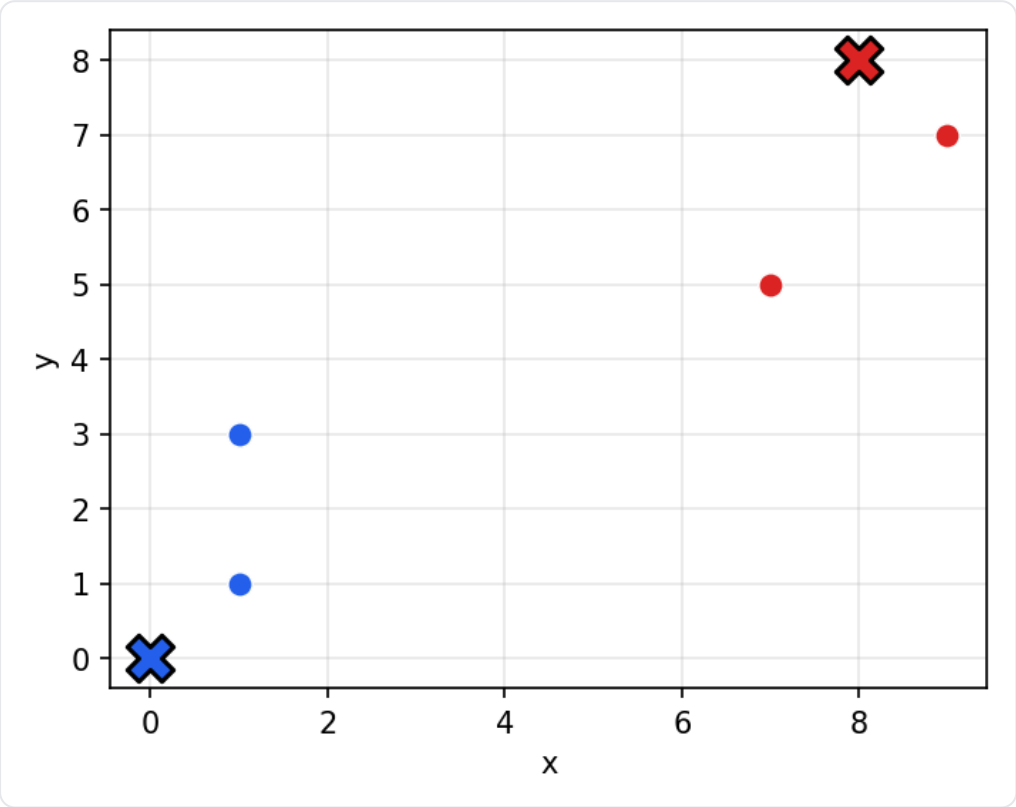
D) The fit is ideal; a near-perfect training R^2 of 0.999 means the model will generalise well.

L12 — Clustering

Q64

one k-means iteration (assign + recompute centroids, 2-D)

You run K-means with K=2 on four points P1=(1,1), P2=(1,3), P3=(7,5), P4=(9,7). The initial centroids are c1=(0,0) and c2=(8,8). After ONE full iteration (assign every point to its nearest centroid by Euclidean distance, then recompute each centroid as the mean of its members), what are the two new centroids?



Four points after the assignment step of iteration 1, coloured by nearest initial centroid (initial centroids c1=(0,0), c2=(8,8) shown as the centroid markers).

A) c1=(1,2) and c2=(8,6)

B) c1=(0,0) and c2=(8,8)

C) c1=(1,1) and c2=(9,7)

D) c1=(4.5,4) and c2=(4.5,4)

Q65 k-means assignment step (nearest centroid, Euclidean)

During the assignment step of K-means there are three centroids: c1=(1,2), c2=(6,5), c3=(3,8). A new point q=(4,4) must be assigned to the cluster of its nearest centroid using Euclidean distance. Which centroid does q join?

A) c1=(1,2)

B) c2=(6,5)

C) c3=(3,8)

D) It is equidistant from all three, so it cannot be assigned.

Q66 distance metrics (Euclidean vs Manhattan)

Two data points sit at a=(0,0) and b=(6,8). A classmate computes the Euclidean distance as 10 and claims the Manhattan (city-block) distance must be the same. What are the Euclidean and Manhattan distances between a and b?

A) Euclidean = 14, Manhattan = 10

B) Euclidean = 10, Manhattan = 10

C) Euclidean = 10, Manhattan = 14

D) Euclidean = 100, Manhattan = 48

Q67 choosing K (elbow method / within-cluster sum of squares)

You run K-means for K=1..6 and record the within-cluster sum of squares (WCSS / inertia): K=1 -> 820, K=2 -> 310, K=3 -> 130, K=4 -> 110, K=5 -> 98, K=6 -> 90. Using the elbow method to choose K, which value should you pick and why?

A) K=6, because it has the lowest WCSS of all the options tested.

B) K=1, because the largest WCSS means the points are best summarised by a single centroid.

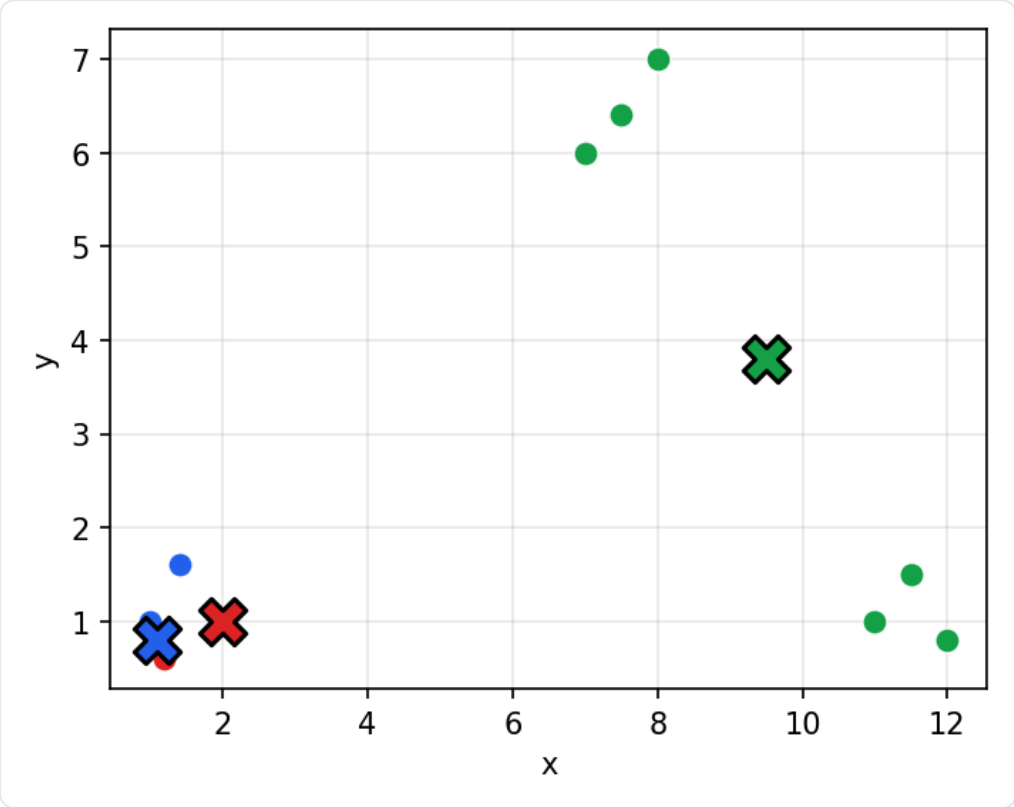
C) K=2, because that is where WCSS first falls below half of the K=1 value.

D) K=3, because WCSS drops steeply up to K=3 and then flattens, putting the 'elbow' at K=3.

Q68

convergence to a local optimum / sensitivity to initialization

The scatter below shows three well-separated round blobs. A run of K-means with K=3 converged to the colouring shown: the left blob has been split between two centroids while the two right blobs were merged under a single centroid, and no point now wants to switch clusters. What does this illustrate about K-means?



A converged but sub-optimal K=3 run: the left blob is split across two centroids while the two right blobs share one centroid.

A) K-means failed to converge and should be left to run for more iterations.

B) K-means converged to a sub-optimal local minimum of SSE because of a poor initial choice of centroids; a different initialization (or multiple runs keeping the lowest SSE) could find the better partition.

C) K-means always finds the globally optimal clustering, so this colouring must be the lowest-SSE partition of the data.

D) This proves K-means cannot be used on data with three clusters; you must use hierarchical clustering instead.

Q69 hierarchical (agglomerative) clustering — single-link merge order

Agglomerative clustering with MIN (single-link) linkage is run on four points with this distance matrix: d(P1,P2)=2, d(P1,P3)=6, d(P1,P4)=10, d(P2,P3)=5, d(P2,P4)=9, d(P3,P4)=4. In what order are the clusters merged, and at what height does the FINAL merge occur?

A) Merge P1&P2 (height 2), then P3&P4 (height 4), then {P1,P2}&{P3,P4} at height 5.

B) Merge P1&P2 (height 2), then P3&P4 (height 4), then {P1,P2}&{P3,P4} at height 10.

C) Merge P3&P4 (height 4) first, then P1&P2 (height 2), then join at height 6.

D) Merge P1&P2 (height 2), then merge that into P3 (height 5), then add P4 at height 4.

Q70 supervised vs unsupervised placement of clustering

A bank has 50,000 customer records WITH NO labels and wants to discover natural groupings of customers from their spending features, then later decide what each group means. Which statement correctly places this task and names a suitable algorithm from this lecture?

A) It is a supervised classification task; train a decision tree on the customer labels.

B) It is neither supervised nor unsupervised; with no labels the data cannot be analysed at all.

C) It is a supervised regression task; fit a line predicting the group number from spending.

D) It is an unsupervised clustering task; K-means can group customers by feature similarity without any labels.

Answer Key & Explanations

L02 — Agents

Q1. **A** — Expected utility is the probability-weighted sum over outcomes: 0.8*10 + 0.2*0 = 8. Option B (10) is the deterministic best case that ignores the 0.2 failure probability, which is the classic 'rationality is not omniscience' trap.

Q2. **B** — Actuators are devices the agent acts WITH (steering, brake); GPS and speedometer are Sensors that feed percepts in, so they are misclassified. Option D confuses the Environment column (what is out there) with the Sensors column (the agent's own measuring devices).

Q3. **C** — The hidden opponent rack makes it partially observable, the ongoing tile draws inject randomness into transitions (stochastic), each move affects later moves (sequential), and there is an opponent (multi-agent). Option D wrongly calls it deterministic, but tile draws DURING play make transitions stochastic.

Q4. **D** — Only randomness in state transitions makes an environment stochastic; a one-shot random initial scramble does not, since the letters do not reshuffle as you solve. Option A applies the wrong test (initial-state randomness), the very confusion the lecture warns against (contrast Scrabble, which keeps drawing tiles).

Q5. **A** — The environment is fully observable and the right action depends only on the current percept, so condition-action rules with no memory (simple reflex) are sufficient. Option B adds internal state that is unnecessary because there is nothing hidden to estimate; choosing a heavier architecture just wastes computation.

Q6. **C** — The table needs a row for every sequence of length 1, 2, and 3: 4^1 + 4^2 + 4^3 = 4 + 16 + 64 = 84. Option B counts only the length-3 sequences and forgets the shorter prefixes that also need rows.

Q7. **D** — The Problem generator proposes exploratory, possibly suboptimal actions so the agent can learn from new experiences. Option A (Critic) only evaluates performance against the external standard and feeds that back; it does not propose the experiment.

L03 — Uninformed Search

Q8. **D** — DFS space is O(b*m): at any moment it stores the current root-to-leaf path and the unexpanded siblings along it, which is linear in depth rather than exponential. A is wrong because O(b^d) is still exponential (the memory killer). B is wrong because DFS is never optimal even with uniform costs (it returns the first goal found, which may be deep/expensive). C is wrong because UCS space is O(b^(1+floor(C*/eps))), exponential; reducing to BFS under uniform costs still gives O(b^d), not O(b*d).

Q9. **A** — FIFO -> shallowest first = BFS; LIFO -> deepest first = DFS; priority queue ordered by path cost g(n) -> cheapest first = UCS; IDS runs a sequence of depth-limited DFS searches, each of which uses a LIFO stack, with only one DFS active at a time. B swaps BFS/DFS. C and D mis-assign UCS (it must use a g(n)-ordered priority queue, not FIFO/LIFO).

Q10. **B** — Enumerate the S-to-G paths: S->A->C->G = 2+4+3 = 9; S->B->C->G = 4+1+3 = 8; S->A->D->G = 2+7+1 = 10. UCS expands nodes in non-decreasing g order and pops C via B (g=5) before C via A (g=6), so it commits to the cheapest route C->G giving 8. The optimal cost is 8 (path S->B->C->G). 9 is the A->C->G route, 10 is the A->D->G route, and 7 is not achievable by any path.

Q11. **B** — With goal-test-on-push, S is expanded and G1 (g=6) is generated first; the test fires and the algorithm returns the 6-cost path before A is ever expanded, so G2 (reachable at cost 2+1=3) does not yet exist on the frontier -- a sub-optimal answer. Goal-test-on-pop defers the test until a node is selected for expansion; UCS then pops A (g=2), expands it to generate G2 (g=3), and pops G2 (g=3) as the cheapest, returning the optimal cost-3 path. A is wrong (the test gives the wrong answer, it is not merely an optimisation). C confuses this with the zero-cost-cycle issue. D misstates the on-push outcome.

Q12. **C** — BFS expands by depth and finds G first via the 2-edge path S->A->G = 1+8 = 9, returning cost 9 (graph-search blocks the later, deeper pushes of G). UCS expands by path cost and finds the 4-edge path S->B->C->D->G = 1+1+1+1 = 4, returning cost 4. So BFS=9, UCS=4: BFS returns the shallowest goal (fewest edges), not the cheapest, and is NOT optimal under non-uniform step costs, whereas UCS is. A swaps the two values; B falsely claims BFS is always optimal; D ignores that the cheaper path is deeper.

Q13. **A** — The IDS work sums to (d+1)b^0 + d*b^1 + ... + 1*b^d, which is O(b^d) -- the same big-O class as BFS, since the last (deepest) level dominates (overhead factor b/(b-1), about 1.11 for b=10, roughly 23% extra at d=5). The real win is space: IDS runs one depth-limited DFS at a time, so it needs only O(b*d) memory versus BFS's O(b^d). B overstates the time cost; C and D invent incorrect bounds.

Q14. **B** — Left-first DFS: pop A and push its children right-to-left (D, C, B) so B is on top. Pop B, push F then E so E is on top; pop E (leaf), pop F (leaf). Pop C, push G; pop G -> goal. Expansion order is A, B, E, F, C, G. A is the BFS (level-order) sequence; C is the mirror-image right-first DFS (A, D, I, H, C, G); D scrambles the order by visiting C before finishing B's subtree.

L05 — Local Search

Q15. **B** — Starting at state 1 (value 3), the right neighbour state 2 (value 5) is higher, so it moves there. From state 2 (value 5) the only neighbours are state 1 (value 3) and state 3 (value 4) — both strictly lower — so hill climbing terminates at state 2, a LOCAL maximum of value 5. The GLOBAL maximum is at state 7 (value 9), separated by the valley at states 4-5, which the greedy climber cannot cross. This is the canonical 'gets stuck at a local maximum' failure.

Q16. **A** — DeltaE = 24 - 20 = 4 (the cost goes UP, a worsening move). For a worsening move the probability is P = exp(-DeltaE/T) = exp(-4/2) = exp(-2) ≈ 0.135. Option B inverts the ratio; option C is only true for improving (downhill-in-cost) moves; option D gives a probability > 1, which is impossible. Equivalently, in maximisation terms Delta = f_next - f_current = -4 and P = exp(Delta/T) = exp(-2) ≈ 0.135 — the same number.

Q17. **C** — For Delta = -3: at T = 100, exp(-3/100) = exp(-0.03) ≈ 0.97, so the downhill move is almost always accepted (broad exploration). At T = 1, exp(-3/1) = exp(-3) ≈ 0.0498, so it is almost never accepted — the algorithm has effectively reduced to greedy hill climbing. As T -> 0+, exp(Delta/T) -> 0 for any worsening move. B reverses the effect of T; C and D contradict the formula (only improving moves are accepted with probability 1).

Q18. **D** — Local search keeps just ONE current state in memory and iteratively modifies it, ignoring path information entirely — there is no frontier or explored set to store, so memory is O(1) regardless of the 1.68 x 10^7 reachable states. BFS, by contrast, must store an exponentially growing frontier. A is false (it can take several steps and may not reach a goal at all); B confuses storing a state with storing its value; C describes population-based GA, not hill climbing (which keeps a single state).

Q19. **C** — Each of the 8 columns has 7 alternative rows (the current row is excluded), giving 8 x 7 = 56 successors evaluated per step. Since h counts attacking pairs and is MINIMISED, h = 12 is better than h = 17 — an improvement of 5 conflicts — so the climber moves to that board. A miscounts successors (56, not 8) and the direction; B uses 8x8 = 64 and a false claim; D wrongly asserts no improvement exists.

Q20. **B** — R = 5 satisfies F_3 = 6 >= 5 while F_2 = 3 < 5, so the smallest i with F_i >= R is i = 3 — R lands in chromosome 3's slice (3, 6]. Roulette-wheel selection picks proportionally to fitness via the cumulative interval, NOT always the fittest (C is the classic 'roulette always picks the best' trap). A invents an unrelated rule; D confuses the random number with an index.

Q21. **A** — Cutting after position 3 splits each parent into a 3-bit prefix and a 7-bit suffix. Child1 = P1[1..3] ++ P2[4..10] = '101' ++ '1011111' = 1011011111. Option C (1000000000) is child2 (P2 prefix '100' + P1 suffix '0000000'). B and D mix the segments incorrectly. Mutation is applied afterward (a separate per-gene bit flip with small probability m), so it does not affect the crossover output itself.

L06 — Adversarial Search

Q22. **C** — Each MIN node takes the minimum of its children. Left MIN = min(5, 9, 4) = 4; right MIN = min(8, 3, 7) = 3. The root MAX then takes the maximum of its MIN children: max(4, 3) = 4. Distractor A grabs the global maximum (ignores the intervening MIN level), B takes the min instead of the max at the root, and D averages (minimax never averages in a deterministic game).

Q23. **A** — The minimax decision at the root is argmax over the children's backed-up values. Left MIN backs up 4, right MIN backs up 3, so MAX chooses the left branch (4 > 3) to maximise its guaranteed worst case. B reverses the logic. C picks the right branch for the wrong reason (a large leaf MIN would never allow). D is false: the move that achieves the root value is specifically the left one.

Q24. **B** — Left MIN = min(2, 9, 7) = 2, so the root sets alpha = 2. Middle MIN reads its first leaf 6 (v = 6, not <= 2, set beta = 6), then its second leaf 1 (v = min(6, 1) = 1). Now v = 1 <= alpha = 2, so the alpha-cutoff fires and the third leaf (value 8) is skipped. Right MIN reads 5, 4, 3 in turn; none ever satisfies v <= 2 before the node finishes, so nothing is pruned there. Only the value-8 leaf is pruned. A is wrong (one leaf is pruned). C and D prune leaves that are actually evaluated.

Q25. **D** — Alpha-beta always returns the exact same minimax value as full minimax regardless of ordering; ordering only affects HOW MANY nodes are expanded. With the worst ordering (worst move first at every node) no cutoffs fire and alpha-beta visits every node, degenerating to O(b^d) like plain minimax. A is the classic error (the answer never changes). B is false (it stays complete). C is backwards (reversing to worst-first hurts, not helps).

Q26. **C** — At an artificial cutoff (game still in progress) the search substitutes Eval(s), a heuristic estimate of the utility for MAX. Because Eval is only an estimate, depth-limited search loses the optimality guarantee unless Eval happens to equal the true Minimax value at every cutoff. A confuses Eval with the rule-given utility U(s), which is only used at TRUE terminals. B is wrong (a value is assigned, just heuristic). D contradicts the whole point of cutting off: the search does not continue past the horizon.

Q27. **A** — Minimax maximises the worst-case payoff: it guarantees at least the minimax value against ANY opponent. If MIN plays sub-optimally (picks a child whose value exceeds the minimum), MAX's realised payoff can only rise, never fall below the minimax value. So MAX gets >= 5. B ignores that real play can exceed the guarantee. C reverses the direction. D is false: minimax still plays validly and safely against a weaker opponent (it just does not actively exploit the weakness).

Q28. **D** — Plain minimax is a DFS of the game tree: worst-case time is O(b^d) (here 4^6) because it expands every node to depth d, while space is only O(b*d) since DFS holds a single root-to-leaf path plus siblings. A ply is one half-move (one player's turn), so d = 6 ply is six half-moves, three per side. A confuses time with space. B confuses space with time. C is wrong on two counts: alpha-beta never changes the backed-up value, though it does share the same O(b^d) worst case.

L07 — Constraint Satisfaction (CSP)

Q29. **A** — The compact n-queens formulation uses n integer variables, one per row, with Xi in {1..n} giving the column of that row's queen (this builds 'one queen per row' into the variable set). Two queens conflict if they share a column (Xi = Xj) or a diagonal (|Xi - Xj| = |i - j|), so the constraint is Xi != Xj AND |Xi - Xj| != |i - j|. B uses Boolean domains (that belongs to the per-cell formulation, not the compact one) and drops the diagonal rule. C mixes the 16-variable per-cell encoding with the wrong {1..4} domain. D keeps only the diagonal constraint and omits the column-difference constraint, so two queens in the same column would be wrongly allowed.

Q30. **B** — At the very start every variable has its full 3-value domain, so MRV ties for all of them and the degree heuristic breaks the tie by picking the variable with the most edges to unassigned neighbours. Reading the graph: SA touches WA, NT, Q, NSW, and V = 5 edges, more than any other node, so SA is selected. T has degree 0 (fewest, not most). WA has degree 2 (WA-NT, WA-SA) and drawing order is irrelevant. NT has degree 3 (WA, SA, Q), fewer than SA's 5.

Q31. **C** — Forward checking removes from each unassigned neighbour every value inconsistent with the just-assigned variable. SA is adjacent to both WA and Q. WA = red strips red from SA, and Q = green strips green from SA, leaving SA = {blue}. It is not yet empty (that wipeout happens only after a later V = blue, per the slide 30-34 trace), so D is wrong; A ignores propagation entirely; B forgets that WA = red also prunes SA.

Q32. **B** — A queen at (3, c) conflicts with the queen at (1, 1) if c = 1 (same column) or if |3 - 1| = |c - 1| (same diagonal), i.e. |c - 1| = 2, which gives c = 3. So columns 1 and 3 are removed, leaving X3 = {2, 4}. A keeps exactly the values that should be removed. C forgets the diagonal at column 3. D is wrong because two legal columns remain.

Q33. **C** — Arc x -> y is consistent only if every x has some supporting y with x < y. x = 1 (y = 2 or 3) and x = 2 (y = 3) are supported, but x = 3 has no y in {1,2,3} with 3 < y, so 3 is pruned from Dx, giving Dx = {1,2}. A is wrong because 3 is unsupported. B prunes the wrong end of the domain (1 is well supported). D confuses direction: enforcing x -> y prunes the tail x's domain, never the head y's (pruning 1 from Dy happens later when enforcing y -> x).

Q34. **D** — MRV picks the unassigned variable with the fewest legal values. V3 is already assigned, so it is not a candidate (eliminating B). V1 has 3 values; V2 and V4 each have 2, so MRV narrows to {V2, V4} and rejects V1 (eliminating A). The degree heuristic breaks the MRV tie by choosing the variable with the MOST constraints on unassigned neighbours: V2 has 3, V4 has 1, so V2 wins (option D). C inverts the degree heuristic (it maximises, not minimises, unassigned-neighbour count).

Q35. **D** — Forward checking only prunes neighbours of the just-assigned variable; arc consistency additionally checks arcs between two unassigned variables (here B and C, which are adjacent in the triangle A-B-C). Test B -> C: B = green is supported by C = blue, B = blue is supported by C = green, so no pruning; by symmetry C -> B is also fine. With both B and C still holding {green, blue}, no domain is forced to a single conflicting value, so arc consistency reports no failure either. Arc consistency would have beaten FC only in the slide-45 situation where two adjacent unassigned variables are both pinned to the same single colour (e.g. both {blue}). A is too strong a claim about why nothing changes but lands on the wrong reasoning chain; B invents an edge effect on D (D borders only C and still has red available); C misstates arc consistency as adding values back -- it only removes unsupported ones.

L09a — Bayesian Networks

Q36. C — Each node contributes one factor P(node | its parents). B and E are roots (priors P(B), P(E)); A has parents {B,E} giving P(A|B,E); J and M each have the single parent A giving P(J|A) and P(M|A). So the joint is P(B)*P(E)*P(A|B,E)*P(J|A)*P(M|A). Option A wrongly drops all conditioning (treats every variable as independent). Option D adds J<->M coupling that the DAG does not contain; given A, John and Mary are conditionally independent.

Q37. A — By the BN factorization, P(A=T,B=T,C=T,D=T) = P(A=T)*P(B=T|A=T)*P(C=T|B=T)*P(D=T|B=T) = 0.4*0.3*0.1*0.95 = 0.0114. Option B (0.114) forgets the P(C=true|B=true)=0.1 factor. Option C (0.012) forgets the P(D=true|B=true)=0.95 factor. Option D (0.0285) forgets the root prior P(A=true)=0.4.

Q38. B — P(+b,~e,+a,~j,+m) = P(+b)*P(~e)*P(+a|+b,~e)*P(~j|+a)*P(+m|+a). Here P(~e)=1-0.002=0.998 and P(~j|+a)=1-0.9=0.1. So = 0.001*0.998*0.94*0.1*0.7 = 6.57 x 10^-5. Option A uses P(+j|+a)=0.9 instead of the complement P(~j|+a)=0.1. Option D is the unrelated false-alarm joint P(+j,+m,+a,~b,~e). Both A and D fail to take the complement for the negated literals.

Q39. D — R's only parent is M, and its only non-descendants here are S, L, and T (R has no descendants of its own). The Markov condition gives R perpendicular {S, L, T} | M. Option B confuses marginal correlation (R and T ARE marginally correlated through M -> L -> T) with conditional independence: once M is fixed, that path no longer carries information to R. Option C wrongly keeps L and T as dependents even though neither is a descendant of R.

Q40. C — Independent parameters per node = 2^(number of parents): B and E (0 parents) need 2^0 = 1 each; A (2 parents) needs 2^2 = 4; J and M (1 parent each) need 2^1 = 2 each. Total = 1 + 1 + 4 + 2 + 2 = 10. Option A (20) counts table CELLS (2^(k+1) per node = 2+2+8+4+4), not independent parameters. Option B (31 = 2^5 - 1) is the parameter count of the UNRESTRICTED full joint. Option D (32 = 2^5) is the number of atomic events.

Q41. B — B and E form a collider (V-structure) B -> A <- E. With no edge between them they are independent a priori: P(B|E)=P(B)=0.001. But conditioning on the common effect Alarm couples the parents: knowing an earthquake occurred 'explains away' the alarm, so P(+b|+a,+e) ~ 0.003 is far below P(+b|+a) ~ 0.37. Option A states the chain/fork rule (conditioning blocks the path) which is exactly backwards for a collider. Option C ignores the explaining-away effect; Option D wrongly claims marginal dependence.

Q42. D — P(+a) = sum over b,e of P(+a|b,e)*P(b)*P(e) = 0.95*0.001*0.002 + 0.94*0.001*0.998 + 0.29*0.999*0.002 + 0.001*0.999*0.998 = 0.0000019 + 0.000938 + 0.000580 + 0.000997 = 0.002517, i.e. about 0.0025. The alarm almost never sounds because burglaries and earthquakes are both rare and the false-alarm rate P(+a|~b,~e)=0.001 is tiny. Option B (0.94) is the single CPT entry P(+a|+b,~e) used without weighting by the priors. Option A and C ignore that the dominant priors P(~b), P(~e) are near 1 while the corresponding alarm probability is only 0.001.

L09b — Hidden Markov Models

Q43. **B** — The Markov assumption gives P(q_t | q_1..q_{t-1}) = P(q_t | q_{t-1}) = a_{q_{t-1} q_t} (state evolution depends only on the previous state), and the output-independence assumption gives P(o_t | all states and obs) = P(o_t | q_t) = b_{q_t}(o_t) (an observation depends only on its own current state). Together with the initial distribution pi multiplied in once at t=1, the joint factors as pi_{q1} b_{q1}(o1) times the product of a_{q_{t-1} q_t} b_{q_t}(o_t). Option A keeps the full history (violates both assumptions), C drops transitions entirely, and D drops the emissions.

Q44. **C** — 0.8 = P(q1 = HOT) is the initial/prior distribution pi_H (probability of STARTING in HOT on day 1). 0.7 = P(q_t = HOT | q_{t-1} = HOT) = a_{HH} is a transition probability (state-to-state). 0.4 = P(o_t = 3 | q_t = HOT) = b_H(3) is an emission/observation probability (state-to-observation). The pitfall is that pi is only the day-1 start probability, not a transition, and emissions map a state to a visible observation, not to another state.

Q45. **A** — Forward initialization multiplies the prior by the emission: alpha_1(Rain) = 0.5*0.9 = 0.45, alpha_1(Sun) = 0.5*0.2 = 0.10. These are JOINT values P(o1, q1), so to get the filtered posterior P(q1 | o1) you normalize by their sum 0.45 + 0.10 = 0.55: Rain = 0.45/0.55 = 0.818, Sun = 0.10/0.55 = 0.182. Option B forgets that the forward variable is a joint and must be normalized to become a belief. Option C wrongly adds instead of multiplies. Option D drops the prior pi.

Q46. **D** — Prediction propagates the belief through the transition matrix: P(tomorrow = Rain) = sum_i P(today = i) * a_{i -> Rain}. Only Rain has nonzero mass today, and the Rain->Rain self-loop in the figure is 0.7, so P(tomorrow = Rain) = 1.0 * 0.7 + 0.0 * 0.4 = 0.7. Option A reads the wrong edge (Rain->Sun = 0.3, which is P(tomorrow = Sun)). Option B ignores the actual numbers. Option C wrongly multiplies the two self-loops.

Q47. **A** — To reach HOT at t=2 you sum over both possible previous states weighted by the transition INTO HOT, then multiply by the emission b_HOT(o2=1)=0.2. The transitions into HOT are a_HH (from HOT) and a_CH (from COLD): [alpha_1(HOT)*a_HH + alpha_1(COLD)*a_CH]*b_H(1) = [0.32*0.7 + 0.02*0.4]*0.2 = 0.232*0.2 = 0.0464. Option B forgets the emission factor. Option C uses transitions INTO COLD (a_HC, a_CC) by mistake. Option D uses max (that is Viterbi, not forward).

Q48. **C** — A belief vector must sum to 1. The forward values are joints, so normalize by the column sum 0.32 + 0.02 = 0.34: HOT = 0.32/0.34 = 0.941, COLD = 0.02/0.34 = 0.059. Option A is wrong because 0.32 + 0.02 = 0.34, not 1. Option B divides by an arbitrary 0.5 instead of the actual sum. Option D normalizes correctly but then incorrectly swaps the two entries, assigning the large mass to the less-likely COLD state.

Q49. **B** — Forward and Viterbi share the same trellis and differ only in the recursion operator: forward uses sum, Viterbi uses max. The forward algorithm sums the joint over all state sequences to give the observation likelihood P(O) = 0.012 + 0.003 = 0.015. Viterbi takes the max to report the probability of the single most-likely sequence, 0.012 (here Q_a). The sum is always at least the max, which is why forward's output (0.015) exceeds Viterbi's (0.012). Options A swaps the two; C and D wrongly make both use the same operator.

L10 — Intro to Machine Learning

Q50. **C** — L10 §3.1 defines reinforcement learning as having no fixed dataset: an agent interacts with an environment defined by states, actions and rewards, gathering its own data by acting to maximise long-term reward. The vacuum's delayed +1/-1 score is a reward signal, not a per-input label, so it is not supervised (A); and it is actively choosing actions to maximise reward rather than just discovering structure in unlabelled inputs, so it is not unsupervised (B). Semi-supervised (D) is not one of the three branches in the lecture.

Q51. **A** — L10 §3.1 says unsupervised learning uses inputs only with no labels, and the learner discovers structure such as clusters. Here the data has no category labels and the goal is to discover groups of similar listeners (the 'sorting laundry without knowing the categories' analogy), which is clustering — the L12 specialisation. It is not supervised (B/C) because there are no (input, output) pairs to learn from, and there is no environment with rewards to make it reinforcement learning (D).

Q52. **D** — L10 §3.2 defines regression as a supervised task whose output is a continuous real value (the 'predicting tomorrow's temperature' analogy is used verbatim in §2). A predicted temperature like 18.7°C is a number, not one of a finite set of discrete classes, so it is regression, not classification (A/B). Clustering (C) is unsupervised and produces groups, not a predicted number.

Q53. **B** — L10 §6.1 / Figure 17 describe overfitting exactly: training error keeps falling toward zero while test error bottoms out and then rises, leaving a large train-test gap (here 0% vs 38%). The fix is to reduce complexity via pre-pruning (max_depth, min_samples_leaf) or post-pruning (§6.3). Underfitting (A) would show large error on BOTH sets. Zero training error is the warning sign of overfitting, not of a perfect model (C). §6.5 stresses that a normal train-test gap is expected from overfitting and does not require a data leak (D).

Q54. **B** — With Spam positive: TP=40 (actual spam predicted spam), FP=10 (actual not-spam predicted spam), FN=20 (actual spam predicted not-spam), TN=130. Precision = TP/(TP+FP) = 40/(40+10) = 40/50 = 0.80. Recall = TP/(TP+FN) = 40/(40+20) = 40/60 ≈ 0.667. Option A swaps the FP and FN terms. Option C computes the metrics for the Not-Spam class instead. Option D confuses precision with accuracy ((TP+TN)/total = 170/200 = 0.85) and miscomputes recall.

Q55. **B** — L10 §3.3 requires the rows used to fit/select the model to be disjoint from the rows used to give the final accuracy estimate; if you choose max_depth by the test set and then report that same test number, the test set has leaked into model selection and no longer estimates performance on unseen data. The correct practice is to tune on a held-out validation set or via cross-validation and reserve the test set for one final unbiased estimate. §6.5 explicitly warns that training error always falls with capacity and is NOT a reliable generalisation estimate, ruling out C and D.

Q56. **A** — L10 §6.4 lists high variance — small data changes change the tree — as a core limitation of single decision trees, and §4.9 introduces ensembles as the remedy: bagging trains many trees on bootstrap samples and majority-votes them so individual variance cancels (the 'many slightly-different experts voting' analogy), with random forest adding a random attribute subset per split. Deeper trees overfit and have MORE variance, not less (B). Gini vs entropy usually pick the same split but do not address variance (C). Training on the test set destroys the disjoint split of §3.3 (D).

L11 — Regression

Q57. **B** — Plug x = 12 into yhat = a + b*x: yhat = 51.849 + 7.527*12 = 51.849 + 90.324 = 142.173. Option C (90.324) is just the slope term 7.527*12 and forgets to add the intercept. Option A treats the spend as 1 unit ($1,000) instead of 12. Option D adds 7.527 once rather than multiplying by 12.

Q58. **C** — Predictions are yhat = 1+2*1=3, 1+2*2=5, 1+2*3=7. Residuals (y - yhat) are 4-3=+1, 4-5=-1, 8-7=+1; squared they are 1, 1, 1; sum of squares (SSE) = 3. MSE = SSE/n = 3/3 = 1. Option A reports SSE (the un-normalised sum) instead of dividing by n=3. Option D squares the summed residual (1) incorrectly or uses the wrong n.

Q59. **A** — At x = 3 the line predicts yhat = 2*3 + 1 = 7. The observed value is 8, so the residual is r = y - yhat = 8 - 7 = +1, which is positive, meaning the point sits above the line. Option B flips the sign (computing yhat - y). Option C forgets to subtract the prediction and just reports the observed y minus nothing meaningful. Option D ignores the gap entirely.

Q60. **C** — R^2 = 1 - SSE/SST. Since every point lies on the line, all residuals are 0, so SSE = 0 and R^2 = 1 - 0/SST = 1. The line passes through every training point and explains all of the variability. Option A is the SSE = SST case (line no better than predicting the mean). For an OLS fit with intercept on training data R^2 is always defined and lies in [0,1], so D is wrong.

Q61. **A** — Too large a learning rate makes each step overshoot the minimum, so the loss oscillates and can diverge rather than decrease; the fix is to lower the learning rate. Option B describes the opposite symptom (slow but steady convergence). Option C is wrong because the OLS objective is convex with a single global minimum (no local minima). Option D is false: gradient descent reaches the same answer as the closed-form solve.

Q62. **B** — Both methods minimise the same SSE objective; because that objective is convex (single global minimum), iterative gradient descent converges to the same coefficients the closed-form normal equations produce directly. Option C reverses which method has hyperparameters: gradient descent needs a learning rate and epochs, the closed-form solve has essentially none. Option D is backwards: the convex objective has no local minima to get stuck in.

Q63. **C** — A very flexible high-degree polynomial drives training error toward zero by fitting the noise, giving an inflated training R^2 but poor generalisation -- the classic overfitting symptom. The remedy is a simpler (lower-degree) model, more data, or regularisation, with model quality judged on held-out test data, not the training fit. Option A is the opposite problem. Option D is the exact trap: a near-perfect training R^2 with poor test behaviour signals overfitting, not good generalisation.

L12 — Clustering

Q64. **A** — Assignment: P1 (dist sqrt(2)=1.41 to c1 vs sqrt(98)=9.9 to c2) and P2 (sqrt(10)=3.16 vs sqrt(74)=8.6) go to cluster 1; P3 (sqrt(74)=8.6 vs sqrt(10)=3.16) and P4 (sqrt(130)=11.4 vs sqrt(2)=1.41) go to cluster 2. Recompute means: c1 = ((1+1)/2,(1+3)/2) = (1,2); c2 = ((7+9)/2,(5+7)/2) = (8,6). Option B is the initial centroids (forgetting to recompute). Option C picks one member per cluster instead of the mean. Option D averages all four points into a single point.

Q65. **B** — dist(q,c1)=sqrt((4-1)^2+(4-2)^2)=sqrt(9+4)=sqrt(13)=3.61; dist(q,c2)=sqrt((4-6)^2+(4-5)^2)=sqrt(4+1)=sqrt(5)=2.24; dist(q,c3)=sqrt((4-3)^2+(4-8)^2)=sqrt(1+16)=sqrt(17)=4.12. The smallest is c2 at 2.24, so q joins c2. The distances differ, so option D is wrong; A and C have larger distances.

Q66. **C** — Euclidean = sqrt((6-0)^2+(8-0)^2) = sqrt(36+64) = sqrt(100) = 10. Manhattan = |6-0|+|8-0| = 6+8 = 14. The two metrics differ here, so the classmate is wrong. Option D forgets the square root on Euclidean (100 is the squared distance) and multiplies the Manhattan terms instead of adding them.

Q67. **D** — The elbow method picks K where adding another cluster stops sharply reducing WCSS. Drops are 820->310 (510), 310->130 (180), 130->110 (20), then 12, 8 — the curve bends at K=3 where the marginal gain collapses, so K=3 is the elbow. Option A chases the global minimum, but WCSS always decreases with K (reaching 0 at K=n) so 'lowest WCSS' over-fits. Option B never clusters. Option C invents a 'half' rule that is not the elbow criterion.

Q68. **B** — K-means does coordinate descent on SSE and is guaranteed to converge, but only to a LOCAL minimum that depends on the initial centroids — exactly the slide-12 'sub-optimal vs optimal' picture. Here it has stabilised (no point switches), so it HAS converged (A is wrong) but to a higher-SSE partition than the clean 3-blob split (C is wrong). Slide-17 remedies — multiple runs keeping the lowest SSE, better seeding, bisecting K-means — fix this; the algorithm is not unusable (D is wrong).

Q69. **A** — Smallest distance is d(P1,P2)=2 -> merge first. Next smallest remaining is d(P3,P4)=4 -> merge. For the final merge under MIN, d({P1,P2},{P3,P4}) = min(d(P1,P3),d(P1,P4),d(P2,P3),d(P2,P4)) = min(6,10,5,9) = 5, so the last merge is at height 5. Option B uses the MAX/complete-link value (10). Option C reverses the first two merges (must always merge the closest pair first). Option D merges out of distance order.

Q70. **D** — Clustering is the unsupervised sibling of classification: no labels are given, and the algorithm groups points so intra-cluster distance is small and inter-cluster distance is large (slide 2). K-means fits exactly. Options A and C both assume labels/targets that do not exist (decision trees and regression are supervised, requiring labels). Option B is wrong because the whole point of unsupervised learning is to find structure when labels are absent.